INVESTIGATION REPORT & IMPLEMENTATION ROADMAP

SellBotLK — Frontend Showcase Investigation & Five-Phase Roadmap

Turning the WhatsApp AI commerce integration into a recruiter-ready web platform

Document	Investigation report and implementation roadmap (read-only audit)
Repository	`SellBotLk` (branch `feature/sprint-15-payment-delivery`, HEAD `ceda9de`)
Date	10 November 2026
Scope	Backend API, admin frontend, tests, CI, configuration (secret values redacted, never viewed or reproduced)
Status	Investigation only. No source, configuration, dependency or database changes were made

Evidence-based audit of the SellBotLK repository. No application code, configuration, dependencies or database were modified.

Table of Contents

0. How to Read This Report

- 0.1 Status legend
- 0.2 Evidence conventions
- 0.3 Naming note

1. Executive Summary

2. Current Architecture

- 2.1 Repository layout
- 2.2 Technologies and versions (actual)
- 2.3 Backend architecture
- 2.4 Database entities and relationships
- 2.5 API endpoint catalogue
- 2.6 Services and business logic
- 2.7 Authentication and authorization
- 2.8 Error handling and logging
- 2.9 Configuration and deployment
- 2.10 Existing frontend: assessment

3. Feature Inventory

- 3.1 Business management
- 3.2 WhatsApp integration
- 3.3 AI capabilities
- 3.4 Security and reliability
- 3.5 Technical capabilities
- 3.6 README claims that differ from the code

4. Feature-to-Page Mapping

5. The WhatsApp Customer Journey (Traced From Code)

- 5.1 Step-by-step trace
- 5.2 Intent handlers (step 10 detail)
- 5.3 Image and payment branch
- 5.4 Incomplete or unverified parts of the journey
- 5.5 How to demonstrate the journey visually
- 5.6 Sample conversations (fictional customers, fictional data)

6. Recommended Website Structure

- 6.1 Three areas, two layouts
- 6.2 Route map
- 6.3 Sidebar (admin layout)
- 6.4 What recruiters should see on the technical side

7. Live Admin Mode and Recruiter Demo Mode

- 7.1 Recommendation
- 7.2 Data-source adapter (frontend architecture)
- 7.3 Privacy and safety rules
- 7.4 Keeping demo data truthful

8. Page Specifications

- A1 Login
- A2 Overview (dashboard home)
- A3 Products
- A4 Product detail
- A5 Orders
- A6 Order detail
- A7 Customers
- A8 Customer detail
- A9 WhatsApp Inbox (with conversation pane)
- A10 AI Playground
- A11 Payment review
- A12 Delivery (Board + Zones)
- A13 Analytics
- A14 Documents & OCR
- A15 Integrations & System status
- R1 Recruiter landing / demo entry
- R2 Customer Journey Player
- R3 Architecture & stack
- R4 API reference
- R5 Security, reliability & testing
- 8.1 Pages that should NOT be created

9. The Five-Phase Implementation Roadmap

- 9.0 Phase dependencies
- 9.1 MVP scope vs optional enhancements
- PHASE 1 — Foundation, Architecture and Design System
- PHASE 2 — Core Business Administration
- PHASE 3 — WhatsApp Inbox and AI Experience
- PHASE 4 — Business Workflows and Analytics
- PHASE 5 — Recruiter Showcase and Production Polish
- 9.2 Effort summary

10. Pinterest → Figma → Frontend Workflow

- 10.1 Process
- 10.2 What to look for in references
- 10.3 Shared vs separate layouts
- 10.4 Per-page Pinterest and Figma guide
- 10.5 Developer handoff requirements (all pages)

11. Backend Changes That May Be Necessary

12. Security and Privacy Considerations

13. Final Recommendations

13.1 Overall website structure
13.2 Navigation and sidebar
13.3 Complete frontend page list
13.4 Pages to design first in Figma
13.5 Exact five-phase roadmap
13.6 Pinterest keywords for the first design stage
13.7 Backend changes
13.8 Frontend architecture recommendation
13.9 Live Admin vs Recruiter Demo strategy
13.10 Security and privacy
13.11 Estimated total effort
13.12 Recruiter walkthrough (2–5 minutes)
13.13 Features that should NOT be built

14. Validation, Assumptions, Risks and Open Items

14.1 Validation performed for this report
14.2 Incomplete or unverified features (consolidated)
14.3 Assumptions
14.4 Key risks

Appendix A — Evidence Index

0. How to Read This Report

0.1 Status legend

Status	Meaning
IMPLEMENTED	Code path exists end to end and is reachable through an endpoint, the webhook, or a background service. Most of these are also covered by tests.
PARTIALLY IMPLEMENTED	Some of the feature exists, but a documented part is missing, unreachable, English-only, or never triggered.
NOT IMPLEMENTED	No working code path. This includes schema-only fields, reserved configuration keys, and features described in the README that have no code.
NOT VERIFIED	Code exists but its behaviour depends on a live external system (Meta, Gemini) or was not executed during this investigation.

0.2 Evidence conventions

- File references are repo-relative, for example `SellBotLk.Api/Services/OrderService.cs:41`.
- Endpoints are written as `METHOD /route`. Every admin route is prefixed with `/api/v1`.
- "Tests" refers to methods in `SellBotLk.Tests/`. The tests were **read, not executed**, because they need a MySQL server (`SELLBOT_TEST_MYSQL`). Pass/fail status is therefore NOT VERIFIED in this report. CI (`.github/workflows/ci.yml`) is configured to run them against MySQL 8.4.
- The working tree had uncommitted changes in five files (`ci.yml`, `README.md`, `Program.cs`, both `.csproj` files) and an untracked `design/` folder when the investigation started. This report describes the working tree **as it is**, including those uncommitted changes.

0.3 Naming note

The brief describes the product as built for **Suhada Furniture**. The string "Suhada" does not appear anywhere in the code, README, or seed data. The code brands itself as **SellBot.lk** (greetings, invoice header, Swagger title). The showcase can describe Suhada Furniture as the client in prose, but the demo data should stay fictional and use the SellBot.lk brand that is actually in the code (see Section 7).

1. Executive Summary

What SellBotLK is today. SellBotLK is an ASP.NET Core 10 Web API with a MySQL database. It receives WhatsApp Cloud API webhooks and uses Google Gemini to classify customer messages and reply in English, Sinhala, or Tamil. It runs real business operations: catalogue search, order creation with transactional stock deduction, rule-based price negotiation, payment-slip OCR with mandatory human approval, PDF invoices, delivery-status notifications, and unpaid-order expiry. A small React 19 + TypeScript admin app (5 pages) covers analytics, orders, order details with payment review, and delivery zones.

Key numbers (verified by reading code)

Item	Count	Evidence
HTTP endpoints	28 (25 admin JWT, 1 login, 2 webhook) + `/health`	`SellBotLk.Api/Controllers/*`, `Webhooks/WhatsAppWebhookController.cs`, `Program.cs:201`
Database tables	11	`SellBotLk.Api/Data/AppDbContext.cs` (README says 9, which is out of date)
Domain services	13 + 2 Gemini integrations	`SellBotLk.Api/Services/`, `SellBotLk.Api/Integrations/` (README says 14)
EF Core migrations	4	`SellBotLk.Api/Data/Migrations/`
Seed data	5 products, 25 Sri Lankan delivery zones	`AppDbContext.cs` `HasData`
Test methods	80 (63 integration on real MySQL + 17 unit)	`SellBotLk.Tests/SecurityAndReliabilityTests.cs`, `UnitTest1.cs`
Admin frontend pages	5 (Login, Dashboard, Orders, Order Details, Delivery Zones)	`sellbotlk-admin/src/pages/`

Strongest material for a showcase

1. A real AI pipeline with deterministic guard rails. Gemini only *classifies*. Prices, stock, order ownership and payment status are enforced in code and proven by tests (`MinimumPriceTests`, `StockConcurrencyTests`, `PaymentReviewTests`, `OrderPrivacyTests`).
2. Payment-slip OCR with human-in-the-loop approval. AI can never mark an order Paid, and a slip or reference can only be used once (unique indexes on `PaymentSlips.ImageSha256` and `Orders.PaymentReference`).
3. Webhook hardening: HMAC-SHA256 signature check, per-sender and global rate limits, idempotency through a `ProcessedWebhookMessages` primary key, and per-message error isolation.
4. Integration tests that host the real API against real MySQL with faked Meta and Gemini upstreams (`SellBotLk.Tests/TestApp.cs`).

Gaps that directly shape the showcase

#	Finding	Impact on the website
G1	**Message text is never stored.** `Conversation` holds only `State`, `Context` (negotiation JSON) and `LastMessageAt`.	A *live* WhatsApp inbox with message history is impossible without new backend work (B4). The inbox must be **demo-only** at first.
G2	**No Customers API.** Customer data is reachable only through orders (`customerName`, `customerPhone`).	Customer pages need a new read endpoint (B1) for Live mode.
G3	** `Customer.TotalOrders`, `TotalSpend` and `LastOrderDate` are never updated** anywhere in the code.	The "Welcome back" greeting (`MessageProcessingService.cs:218`) can never fire, and customer statistics must be computed from orders.

#	Finding	Impact on the website
G4	**Interactive button replies are not handled.** Track Order and Cancel Order buttons are *sent* (`MessageProcessingService.cs:680`), but an inbound `interactive` message falls through to "couldn't process this type" (`WhatsAppWebhookController.cs:202`).	The demo must not show the buttons working until B7 is done.
G5	**WhatsApp orders never capture a delivery area or address**, and delivery fees are never added to order totals.	The delivery board will show empty areas for WhatsApp orders. Do not imply checkout includes delivery.
G6	**Several replies are English-only** (order confirmation, payment-slip replies, delivery notifications, expiry fallback for some paths).	Multilingual support is PARTIAL. Label it honestly in demos.
G7	**Schema-only features:** fraud flags (`IsFraudFlagged`, `FraudPending` are never set), `DailyReports` (never written), `InventoryLogs` (written but not exposed), Hangfire (configuration keys only, no package).	Do not build pages for them. Hide the fraud badge or label it "not active".
G8	**Operations gaps:** `/health` does not check the database, CORS is hard-coded to `localhost:5173`, Swagger is Development-only, `wwwroot` images are not served (no `UseStaticFiles`), and there is no deployment configuration.	Phase 5 backend tasks before any public Live deployment.

Recommendation in one paragraph. Extend the existing `sellbotlk-admin` app rather than starting over. Its stack (React 19, React Router 7, Vite 8, TypeScript 6, Recharts 3) is current. Restructure it into two layouts: an **Admin layout** with a sidebar for business pages, and a **Presentation layout** for recruiter pages. Add a **data-source adapter** so every page runs against either the real API (Live Admin Mode) or bundled fictional fixtures (Recruiter Demo Mode). Deploy the public recruiter site as a **demo-only build** that contains no API URL, so it physically cannot reach production data. Build in five phases: foundation, business administration, WhatsApp and AI experience, workflows and analytics, then showcase polish. Seven small backend additions (B1–B7) unlock the live versions of pages that are demo-only today.

2. Current Architecture

2.1 Repository layout

Path	Contents
`SellBotLk.Api/`	ASP.NET Core 10 Web API: Controllers (6), Webhooks (1), Services (13), Integrations (2 Gemini clients), Middleware (HMAC), Data (DbContext, 2 repositories, 4 migrations), Models (11 entities, DTOs, 1 enum). `Extensions/` and `Jobs/` are empty folders.
`SellBotLk.Tests/`	xUnit tests. `TestApp.cs` hosts the real API on MySQL with a `FakeHandler` for Meta and Gemini.
`sellbotlk-admin/`	React + Vite admin SPA (5 pages, 3 shared component files, typed API client).
`.github/workflows/ci.yml`	Two jobs: admin build + lint, and API build + test with a MySQL 8.4 service.
`design/`	Untracked folder with 13 reference images. Not analysed here, so this report does not pre-empt the visual direction.

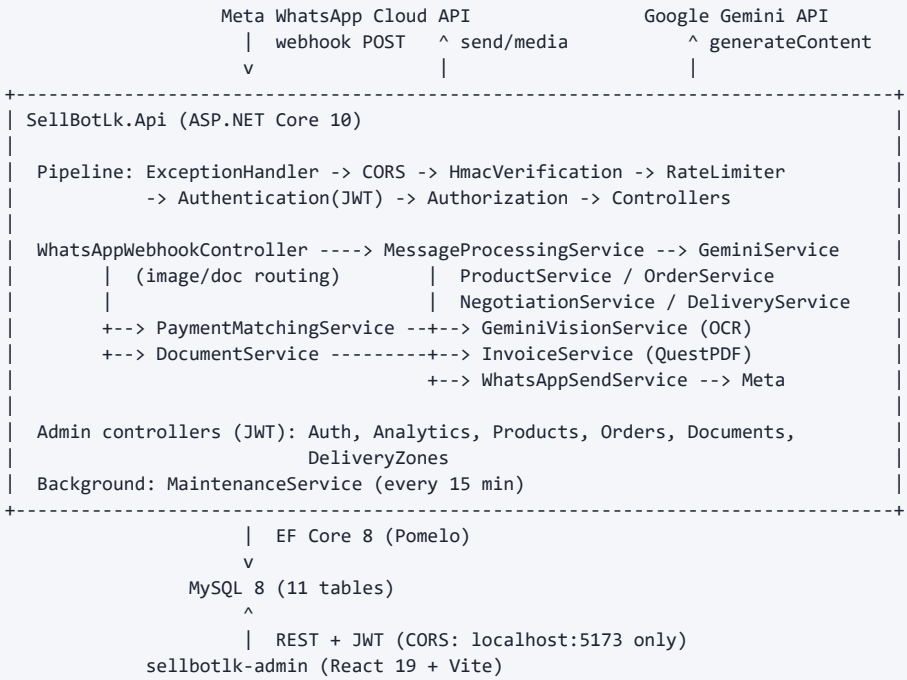
2.2 Technologies and versions (actual)

Layer	Technology	Version (from manifests)
Runtime	.NET	`net10.0` (`SellBotLk.Api.csproj`)
Web framework	ASP.NET Core (controllers)	10
ORM	EF Core + Pomelo MySQL provider	EF Core 8.0.33 (Relational pinned), Pomelo 8.0.3, MySqlConnector 2.6.2
Database	MySQL	8.x (CI uses `mysql:8.4`)
Auth	JWT Bearer	`Microsoft.AspNetCore.Authentication.JwtBearer` 10.0.14
API docs	Swashbuckle	10.3.1 (Swagger UI in Development only)
PDF	QuestPDF (invoices), PdfPig (PDF page count only)	2026.2.4, 0.1.9
Config	DotNetEnv (`.env` loader)	3.1.1
Health	`AspNetCore.HealthChecks.MySql`	8.0.1 (**referenced but not used**)
AI	Google Gemini REST `v1beta generateContent`	Model from config (`.env.example`: `gemini-2.5-flash`; code fallback `gemini-1.5-flash`)
Messaging	Meta WhatsApp Cloud API (Graph)	`v22.0` (hard-coded in `WhatsAppSendService.cs`, `MediaDownloadService.cs`)
Frontend	React, React DOM	^19.2.4
Routing	react-router-dom	^7.14.1
Charts	Recharts	^3.8.1
Build	Vite, @vitejs/plugin-react	^8.0.4, ^6.0.1
Language	TypeScript	~6.0.2
Lint	ESLint 9 + typescript-eslint 8 + react-hooks 7	as in `package.json`
Tests	xUnit 2.5.3, Moq 4.20.70, FluentAssertions 6.12.0, Mvc.Testing 10.0.14, EF InMemory 8.0.33	`SellBotLk.Tests.csproj`
CI	GitHub Actions (Node 20, .NET 10.0.x)	`.github/workflows/ci.yml`

There is **no** frontend test framework, UI component library, data-fetching library, CSS framework, or state-management library. Styling is plain global CSS (`src/index.css`, 311 lines). `src/App.css` (184 lines) is not imported anywhere and is leftover Vite template code.

2.3 Backend architecture

A layered monolith: Controllers → Services → Repositories / `AppDbContext` → MySQL. Outbound integrations use typed `HttpClient`s. All services are scoped, except `InvoiceService` (singleton) and `MaintenanceService` (hosted).



Middleware order (`SellBotLk.Api/Program.cs:179-201`): exception handler → CORS → `HmacVerificationMiddleware` (runs first, so unsigned traffic cannot consume the webhook rate limit) → rate limiter → authentication → authorization → controllers → `/health`.

2.4 Database entities and relationships

Table	Purpose	Key fields	Relationships / indexes	Used?
`Products`	Catalogue	Name, Category, Price, **MinPrice**, BulkDiscountPercent, BulkMinQty, StockQty, LowStockThreshold, Color, Material, Style, ImageUrl, IsActive	idx Category, IsActive	Yes
`Customers`	WhatsApp customers	**PhoneNumber (unique)**, Name, PreferredLanguage, TotalOrders, TotalSpend, LastOrderDate, IsBlocked, Notes	1–N Orders, Documents, Conversations	Partly. Counters and `IsBlocked` are never written or enforced.
`Orders`	Orders	OrderNumber (unique, `ORD-{yyyy}-{id:D3}`), Status, PaymentStatus, **PaymentReference (unique)**, TotalAmount, DiscountAmount, DeliveryAddress, DeliveryArea, DriverNote, IsFraudFlagged, FraudReason	FK Customer (Restrict); idx Status, CreatedAt, CustomerId	Yes. Fraud fields are never set.
`OrderItems`	Lines	Quantity, UnitPrice, NegotiatedPrice	FK Order (Cascade), Product (Restrict)	Yes

Table	Purpose	Key fields	Relationships / indexes	Used?
`PaymentSlips`	Slip images for review	**ImageSha256 (unique)**, ImageBytes (longblob), MimeType	FK Order (Cascade)	Yes
`Conversations`	Per-customer bot state	State (Greeting/Browsing/Ordering/Negotiating/Support/Completed), Context (JSON), LastMessageAt	FK Customer (Cascade)	Yes. **No message text.**
`Documents`	OCR results	Type (CustomerBill/SupplierInvoice/PaymentSlip/DamageReport), ExtractedData (JSON), VendorName, TotalAmount, IsProcessed, ProcessingNotes	FK Customer (SetNull)	Yes
`InventoryLogs`	Stock audit	ChangeType (OrderDeduction/SupplierRestock/ManualAdjust/Return), QuantityBefore/After, ReferenceId	FK Product (Restrict)	Written, but no read endpoint
`DeliveryZones`	25 seeded zones	ZoneName, DeliveryFee, EstimatedDays, FreeDeliveryThreshold, IsActive	—	Yes (read-only)
`DailyReports`	Planned daily summary	ReportDate (unique), TotalOrders, TotalRevenue, TopProductId, ReportText, SentAt	FK Product (SetNull)	**Never written**
`ProcessedWebhookMessages`	Idempotency	MessageId (PK), ProcessedAt	idx ProcessedAt; purged after 30 days	Yes

```
Customers 1---* Orders 1---* OrderItems *---1 Products 1---* InventoryLogs
    |             |
    |             1---* PaymentSlips
    1---* Conversations (state + negotiation JSON only)
    1---* Documents (OCR output)
DailyReports *---0..1 Products      DeliveryZones (standalone)
ProcessedWebhookMessages (standalone, dedupe)
```

Order state model (actual code). `OrderService.CreateOrderAsync` creates every order directly as `Confirmed` + `Unpaid` (`OrderService.cs:41`). `Pending` is never produced by current code paths.

```
Fulfilment (PUT /orders/{id}/delivery-status, DeliveryService.IsValidTransition):
  Confirmed -> Processing -> Dispatched -> Delivered
  Pending/Confirmed/Processing/FraudPending -> Cancelled (via CancelOrderAsync, stock restored once)

Payment (PaymentMatchingService):
  Unpaid --(slip OCR, amount within 1%, slip unused)--> PendingReview
  PendingReview --(admin approve)--> Paid  (+ WhatsApp confirmation + PDF invoice)
  PendingReview --(admin reject)--> Unpaid (slip released, customer told)
  Unpaid + Confirmed/Pending for 24h --(MaintenanceService)--> Cancelled (stock released)
```

2.5 API endpoint catalogue

#	Method & route	Auth	Purpose	Used by current frontend
1	`POST /api/v1/auth/login`	Rate-limited 5/min/IP	Returns a 24-hour JWT (`AuthController.cs`)	Yes
2	`GET /api/v1/analytics/summary?from&to`	JWT	KPIs, revenue by day, top 5 products, status counts, 5 recent orders	Yes (Dashboard)

#	Method & route	Auth	Purpose	Used by current frontend
3	`GET /api/v1/products`	JWT	Active products	No
4	`GET /api/v1/products/{id}`	JWT	Product by id	No
5	`GET /api/v1/products/low-stock`	JWT	Products at or below their threshold	No
6	`POST /api/v1/products`	JWT	Create product (validated DTO)	No
7	`PUT /api/v1/products/{id}`	JWT	Partial update	No
8	`DELETE /api/v1/products/{id}`	JWT	Soft delete (`IsActive=false`)	No
9	`POST /api/v1/products/search`	JWT	**Gemini smart search**, also returns the formatted WhatsApp message	No
10	`POST /api/v1/products/visual-search`	JWT	**Gemini Vision** photo → attributes → weighted match (multipart)	No
11	`GET /api/v1/orders?status&customerId`	JWT	List orders (no pagination, search, or payment filter)	Yes
12	`GET /api/v1/orders/{id}`	JWT	Order detail	Yes
13	`GET /api/v1/orders/number/{orderNumber}`	JWT	Lookup by number	No
14	`POST /api/v1/orders`	JWT	Manual order (same rules as WhatsApp)	No
15	`PUT /api/v1/orders/{id}/status`	JWT	Generic status change (**no transition table, no customer notification**)	Yes
16	`PUT /api/v1/orders/{id}/delivery-status`	JWT	Validated fulfilment transition + WhatsApp notification + driver note	Yes
17	`PUT /api/v1/orders/{id}/payment/approve`	JWT	PendingReview → Paid, sends confirmation + PDF invoice	Yes
18	`PUT /api/v1/orders/{id}/payment/reject`	JWT	PendingReview → Unpaid with reason	Yes
19	`GET /api/v1/orders/{id}/payment-slip`	JWT	Slip bytes (`nosniff`)	Yes
20	`PUT /api/v1/orders/{id}/cancel`	JWT	Cancel + restore stock exactly once	Yes
21	`GET /api/v1/documents?type&page&pageSize`	JWT	Paged OCR documents (**returns raw entity, including extracted JSON**)	No
22	`GET /api/v1/documents/{id}`	JWT	Document detail	No
23	`POST /api/v1/documents/process`	JWT	Upload invoice, slip or damage photo for Gemini Vision (≤10 MB, JPEG/PNG/WebP/PDF)	No
24	`GET /api/v1/delivery-zones`	JWT	Active zones	Yes
25	`POST /api/v1/delivery-zones/check`	JWT	Serviceability + fee, Gemini fuzzy area match	Yes
26	`GET /api/v1/webhook/whatsapp`	Verify token	Meta subscription handshake	—
27	`POST /api/v1/webhook/whatsapp`	HMAC + rate limit	Inbound messages	—

#	Method & route	Auth	Purpose	Used by current frontend
28	`GET /health`	None	Liveness only (no database check)	No

Response envelope: `{ success, data, count?, message?, error? }`. Errors are `{ success:false, error }`. Unhandled exceptions return RFC 9110 problem JSON.

2.6 Services and business logic

Service	Responsibility	Key methods
`MessageProcessingService`	Text and image conversation brain, intent routing	`ProcessTextMessageAsync` (`:42`), `ProcessImageMessageAsync` (`:176`), handlers for each intent, `FindBestProductMatch` (`:808`)
`GeminiService`	Intent/language parse, smart-search calls, image attributes	`ParseMessageAsync` (`:36`, 10-minute cache), `CallGeminiAsync` (`:268`, 3 attempts with backoff), JSON repair, fallback intent
`GeminiVisionService`	OCR prompts for invoices, slips, and damage photos	`ExtractInvoiceDataAsync`, `ExtractPaymentDataAsync`, `ExtractDamageDataAsync`
`OrderService`	Order creation (transaction, ordered row locks), cancel, expiry, confirmation text	`CreateOrderAsync` (`:41`), `CancelOrderAsync`, `ExpireUnpaidOrdersAsync` (`:312`)
`PaymentMatchingService`	Slip OCR → match → claim → review → approve/reject → invoice	`ProcessPaymentSlipAsync` (`:61`), `ApprovePaymentAsync` (`:295`), `RejectPaymentAsync` (`:324`)
`NegotiationService`	Accept / counter / reject against `MinPrice` and bulk tiers; context stored in `Conversation.Context`	`EvaluateOfferAsync` (`:29`)
`DeliveryService`	Zone check (exact match, then Gemini fuzzy match), fulfilment transitions, notifications	`CheckZoneAsync` (`:40`), `UpdateDeliveryStatusAsync` (`:121`)
`ProductService`	CRUD, Gemini smart search with text fallback, WhatsApp formatting	`SmartSearchAsync`, `BuildCatalogueSummaryAsync`, `FormatProductsForWhatsApp`
`VisualSearchService`	Weighted attribute score (type 40, colour 25, material 20, style 15)	`SearchByImageAsync` (`:23`)
`DocumentService`	Owner-only supplier invoices → restock with audit log; damage reports	`ProcessDocumentAsync`, `IsOwnerPhone`
`InvoiceService`	QuestPDF invoice	`GenerateInvoicePdf`
`WhatsAppSendService`	Text, buttons, image, document upload, read + typing indicator	`Send*Async`, `MarkAsReadWithTypingAsync`
`MediaDownloadService`	Two-step Meta media download, MIME allow-list	`DownloadMediaAsync`, `IsAcceptedType`
`AnalyticsService`	In-memory aggregation over the date range	`GetSummaryAsync`
`MaintenanceService`	`BackgroundService`, every 15 minutes: expire unpaid orders, purge dedupe rows older than 30 days	`ExecuteAsync` (`:29`)

2.7 Authentication and authorization

- **Single admin account** from configuration (`Admin:Username` / `Admin:Password`). There is no user table and no roles beyond the `Admin` claim. The app **refuses to start** if the password is shorter than 12 characters or if `Jwt:Secret` is shorter than 32 bytes (`Program.cs`).
- Login uses constant-time comparison (SHA-256 then `FixedTimeEquals`, with `&` so both comparisons always run), a 24-hour HS256 JWT (issuer `SellBotLk`, audience `SellBotLk.Admin`), and a rate limit of 5 attempts per minute per IP.

- Every admin controller has `[Authorize]`. There is no role or policy granularity, and none is needed for a single owner.
- Frontend: the token is stored in `localStorage`, and a 401 clears it and redirects to `/login` (`sellbotlk-admin/src/api/http.ts`). There is no refresh token and the UI does not handle expiry.

2.8 Error handling and logging

- Global `UseExceptionHandler` returns a generic problem JSON with no stack traces.
- Webhook: per-message `try/catch`, `ChangeTracker.Clear()` after a failure, a best-effort "Sorry, something went wrong" reply, and HTTP 500 only for failures before any message was claimed, so Meta can retry.
- `MessageProcessingService` wraps each message in a top-level safety net, so the customer always gets a reply.
- Gemini: 3 attempts with exponential backoff (1s, 2s), checks for SAFETY/RECITATION blocks, skips "thought" parts on 2.5 models, repairs truncated JSON, and falls back to intent `Other`.
- Logging uses the default `ILogger` console provider with structured templates. Phone numbers are masked (`94771***123` style) in webhook and message logs. There is no Serilog, central log store, or correlation IDs.
- `WhatsAppSendService.PostToWhatsAppAsync` **logs but does not throw** on non-2xx responses, so a send failure is invisible to callers.

2.9 Configuration and deployment

- Configuration comes from `appsettings.json` (empty secrets), `appsettings.Development.json` (git-ignored), and `.env` loaded by `LegacyEnvConfiguration` (maps flat names such as `GEMINI_API_KEY` to nested keys).
- There is no Dockerfile, IaC, hosting config, or production `appsettings`. CORS allows only `http://localhost:5173` and `http://127.0.0.1:5173`. Swagger is enabled only in Development. The rate limiter partitions by remote IP with no `UseForwardedHeaders` (noted in a code comment).
- The Gemini API key is passed as a URL query parameter (`?key=`). Moving it to the `x-goog-api-key` header would keep it out of any URL logging.

2.10 Existing frontend: assessment

Aspect	Current state	Verdict
Stack	React 19, RR 7, Vite 8, TS 6, Recharts 3	Current. **Keep.**
Routing	Flat routes in `App.tsx`, `RequireAuth` wrapper, `*` → `/`	Extend with route groups (`/app/*`, `/showcase/*`, `/`).
Layout	Top navigation bar (`AppLayout.tsx`, 3 links + logout)	Replace with a sidebar admin layout and add a presentation layout.
API client	Hand-written `fetch` helpers + `sellbotApi` object + hand-written types (`types.ts`)	Good base. Put it behind a **data-source interface** so a demo implementation can be swapped in.
Components	`Card`, `Badge`, `Button`, `Input`, `Select`, `Field`, `Table`, `StatCard`	Small but reusable. Evolve them into the design system.
State/data fetching	`useState` + `useEffect` per page, with manual loading/error states	Acceptable at this size. A shared `useAsync` hook removes repetition. No library is required.
Styling	One global CSS file, one `@media` query	Replace with design tokens (CSS custom properties) from the Figma file. Responsive work is needed.
Tests	None for the frontend	Add Vitest + Testing Library only if you want frontend tests (Phase 5, optional).
Leftovers	`App.css`, `assets/hero.png`, `react.svg`, `vite.svg`, `<title>sellbotlk-admin</title>`	Remove or replace in Phase 1.

Conclusion: the frontend can be extended. It needs a structural refactor (layouts, route groups, data-source adapter, tokens) before new pages are added, but no rewrite and no new framework.

3. Feature Inventory

Columns: **Status** · **What / how** · **Evidence** · **Endpoints** · **Existing frontend** · **Visual demo?** · **Backend work for the showcase**.

3.1 Business management

Feature	Status	What / how	Evidence	Endpoints	Existing FE	Visual demo?	Backend work
Product catalogue CRUD	IMPLEMENTED	Create/partial update/soft delete; validated DTOs (`[Range]`, `[MaxLength]`); MinPrice and bulk tiers stored per product	`ProductsController.cs`, `ProductService.cs`, `ProductRepository.cs`	#3–8	None	Yes: table, drawer, edit form	None
Low-stock alerts	IMPLEMENTED	`StockQty <= LowStockThreshold`	`ProductRepository.GetLowStockAsync`	#5	None	Yes: badge, widget	None
Inventory audit log	PARTIALLY IMPLEMENTED	Every deduction, restock and return is logged with before/after quantities and a reference	`ProductRepository.LogStockChange`, test `CreatedOrder_HasRealNumber_ProductNames_AndAuditLog`	none	None	Yes, if exposed	B9 (optional read endpoint)
Customers	PARTIALLY IMPLEMENTED	Auto-created from WhatsApp sender; language preference updated per message; counters never updated; `IsBlocked` not enforced	`MessageProcessingService.GetOrCreateCustomerAsync` (`:894`)	none (only via orders)	None	Yes: list + detail	**B1** (read API), **B2** (counters)
Orders: create	IMPLEMENTED	One transaction; ordered row locks; conditional stock decrement (`StockQty >= qty`); bulk discount floored at MinPrice; real number from DB id	`OrderService.CreateOrderAsync` (`:41`), `ProductRepository.TryDeductStockAsync`; `StockConcurrencyTests`	#14, WhatsApp	None for create	Yes	None
Orders: list/detail	IMPLEMENTED	Filter by status and customerId only	`OrderRepository.GetAllAsync`	#11–13	`OrdersPage`, `OrderDetailsPage`	Yes	B8 optional (pagination, search, payment filter)
Order status changes	PARTIALLY IMPLEMENTED	Delivery path is validated and notifies the customer. The generic `/status` path skips the transition table and sends no notification.	`DeliveryService.IsValidTransition` (`:235`), `OrderService.UpdateStatusAsync`	#15, #16	Both exposed in `OrderDetailsPage`	Yes: stepper	B12 (align or retire `/status`)
Cancellation + stock restore	IMPLEMENTED	Single conditional update means only one caller wins; restock with an audit log	`OrderService.CancelOrderAsync`; `CancellationTests`	#20	Yes	Yes	None

Feature	Status	What / how	Evidence	Endpoints	Existing FE	Visual demo?	Backend work
Unpaid order expiry	IMPLEMENTED	Every 15 minutes, cancels Unpaid Pending/Confirmed orders older than `Orders:UnpaidExpiryHours` (default 24) and notifies the customer in si/ta/en	`OrderService.ExpireUnpaidOrdersAsync` (`:312`), `MaintenanceService`	none	None	Yes: countdown chip	None
Payment slip review	IMPLEMENTED	OCR → match own unpaid order within 1% → claim (unique image hash and reference) → PendingReview → admin approve/reject	`PaymentMatchingService.cs`; `PaymentReviewTests` (13 tests)	#17–19, #23	Panel in `OrderDetailsPage`	Yes: review queue	None (B8 payment filter optional)
PDF invoices	IMPLEMENTED	QuestPDF, sent as a WhatsApp document after approval	`InvoiceService.cs`, `PaymentMatchingService.cs:465`	(side effect of #17)	None	Yes: sample PDF preview (demo)	Optional: `GET /orders/{id}/invoice`
Delivery zones	IMPLEMENTED (read-only)	25 seeded zones; free-delivery thresholds; Gemini fuzzy area match	`DeliveryService.CheckZoneAsync` (`:40`)	#24, #25	`DeliveryZonesPage`	Yes	None (no zone CRUD; do not build one)
Delivery tracking	PARTIALLY IMPLEMENTED	Status transitions, driver note and WhatsApp notifications exist. No delivery area or address is captured in WhatsApp orders, and the fee is never added to totals.	`DeliveryService.UpdateDeliveryStatusAsync` (`:121`), `CreateOrderDto`	#16	Partial (select + note)	Yes: board	None for the board; G5 is a product gap
Supplier invoice → restock	IMPLEMENTED	Owner phone or admin upload only; Gemini extracts line items; partial name match; transaction; owner summary	`DocumentService.ProcessSupplierInvoiceAsync`; `SupplierInvoiceTests`	#21–23, WhatsApp document from owner	None	Yes: upload + extracted table	B10 (DTO instead of raw entity)
Damage reports	PARTIALLY IMPLEMENTED	Vision prompt + owner alert exist, but they are reachable **only via the admin upload**. WhatsApp documents route to slip or invoice only.	`DocumentService.cs`, `WhatsAppWebhookController.cs:324`	#23	None	Demo only	None
Customer bills (`CustomerBill` type)	NOT IMPLEMENTED	Falls to "Unknown document type"	`DocumentService.cs` default branch	—	—	No	—
Fraud detection	NOT IMPLEMENTED	Fields exist and are displayed, but nothing sets them	`Order.IsFraudFlagged`, `OrderStatus.FraudPending`	—	Badge in `OrderDetailsPage`	**Do not demo**	—
Daily reports	NOT IMPLEMENTED	Table only	`Dailyreport.cs`	—	—	No	—
Business information (bank details, owner phone)	IMPLEMENTED (config only)	Read from configuration and shown in order confirmations	`OrderService.FormatOrderConfirmationMessage` (`:359`)	none	None	Static, masked	B6 exposes only "configured: yes/no"

3.2 WhatsApp integration

Feature	Status	What / how	Evidence	Endpoints	Visual demo?	Backend work
Webhook verification handshake	IMPLEMENTED	`hub.mode=subscribe` + verify token → echo challenge	`WhatsAppWebhookController.Verify`	#26	Diagram only	None
Inbound processing	IMPLEMENTED	Text, image, document, audio (placeholder) and default branches; sender name from contacts	`WhatsAppWebhookController.Receive` (`:80`), `ProcessMessageAsync` (`:172`)	#27	Yes: pipeline animation	None
Status callbacks (sent/delivered/read)	PARTIALLY IMPLEMENTED	Deserialised (`WhatsAppStatusDto`) and then ignored	`WhatsAppMessageDto.cs`	#27	No	Optional (B4 could store them)
Outgoing text / image / document	IMPLEMENTED (NOT VERIFIED live)	Graph `v22.0 /messages`; documents uploaded via `/media` first	`WhatsAppSendService.cs`	—	Yes: chat bubbles	None
Interactive buttons (Track/Cancel)	PARTIALLY IMPLEMENTED	Sent with order confirmations; **replies unhandled** (falls to default branch)	`MessageProcessingService.cs:680`, webhook `default:` (`:202`)	—	Show as sent only	**B7**
Read receipt + typing indicator	IMPLEMENTED (NOT VERIFIED live)	`status=read + typing_indicator` before processing	`WhatsAppSendService.MarkAsReadWithTypingAsync`	—	Yes: "typing…"	None
Context-aware image routing	IMPLEMENTED	An image is treated as a payment slip if the conversation is in Ordering state or the customer has an unpaid Confirmed/Processing order; otherwise visual search	`HandleImageMessageAsync` (`:227`), `IsPaymentContextAsync` (`:340`)	—	Yes: decision diamond	None
Document gatekeeping	IMPLEMENTED	Owner → supplier invoice; customer with an open order → slip; otherwise refused before any processing	`HandleDocumentMessageAsync` (`:278`); `SupplierInvoiceTests`	—	Yes	None
Media download	IMPLEMENTED	Two-step Graph download with bearer token; MIME allow-list	`MediaDownloadService.cs`	—	Diagram	None
Voice notes	NOT IMPLEMENTED	Replies "Audio processing coming soon"	webhook `case "audio"`	—	No	—
Conversation management	PARTIALLY IMPLEMENTED	One `Conversation` per customer; state machine; stale reset after 2 hours; negotiation context JSON. **No transcript, no agent handover, no admin reply.**	`GetOrCreateConversationAsync` (`:918`)	none	Demo only (live needs B3/B4)	**B3**, **B4**
Message templates (WhatsApp HSM)	NOT IMPLEMENTED	Only free-form session messages are sent. Proactive notifications outside the 24-hour window would need approved templates (Meta policy; NOT VERIFIED against current Meta rules).	—	—	No	Out of scope

3.3 AI capabilities

Feature	Status	What / how	Evidence	Visual demo?	Backend work
Gemini text integration	IMPLEMENTED (NOT VERIFIED live)	REST `generateContent`, temperature 0.3, 2048 tokens, retries, safety handling	`GeminiService.CallGeminiAsync (:268)`	Yes	None
Intent detection (10 intents)	IMPLEMENTED	Greeting, ProductSearch, Order, Reorder, OrderStatus, PriceNegotiation, Complaint, DeliveryInfo, PaymentConfirmation, Other; returns confidence; catalogue injected in prompt	`GeminiService.BuildParsePrompt`, `MessageProcessingService.cs:94`	Yes: intent chip + confidence	**B5** for live playground
Language detection (en/si/ta)	IMPLEMENTED	Same Gemini call; stored as `Customer.PreferredLanguage`; mixed "Singlish" is explicitly prompted (`chair ekak denna`)	`GeminiService.cs`, `MessageProcessingService.cs:72`	Yes: language chip	None
Sinhala replies	PARTIALLY IMPLEMENTED	Hand-written si templates for greeting, search, status, delivery info, complaint, payment prompt, negotiation, order errors, reorder, expiry. **English-only**: order confirmation, slip results, payment approved, delivery notifications, image analysis, generic errors.	`MessageProcessingService.cs`, `OrderService.cs:359`, `PaymentMatchingService.cs`	Yes, labelled	Optional i18n of the remaining templates
Tamil replies	PARTIALLY IMPLEMENTED	Same coverage as Sinhala	as above	Yes, labelled	as above
English replies	IMPLEMENTED	Default for every template	—	Yes	None
Product recommendation / smart search	IMPLEMENTED	Gemini picks the top 3 product IDs from the catalogue; falls back to text match if parsing fails; product images sent individually	`ProductService.SmartSearchAsync`, `HandleProductSearchAsync (:782)`	Yes	None (endpoint #9 exists)
Visual product search	IMPLEMENTED	Gemini Vision → {type, colour, material, style} → weighted score → top 3	`GeminiService.AnalyzeImageAsync`, `VisualSearchService.cs`	Yes	None (endpoint #10 exists)
OCR: payment slips	IMPLEMENTED	Bank, amount, reference, date, names, confidence. Below 0.5 confidence asks for a clearer photo.	`GeminiVisionService.ExtractPaymentDataAsync`	Yes	None
OCR: supplier invoices	IMPLEMENTED	Vendor, line items, totals; PDFs passed natively to Gemini	`GeminiVisionService.ExtractInvoiceDataAsync`, `DocumentService.cs`	Yes	B10
OCR: damage photos	PARTIALLY IMPLEMENTED	Admin upload only	see 3.1	Demo	None
AI-generated free-text replies	PARTIALLY IMPLEMENTED	Only the `Other` intent uses Gemini's `replyMessage`; every business reply is a deterministic template	`MessageProcessingService.cs:140`	Yes: "template vs AI" toggle	None
Fuzzy delivery-area matching	IMPLEMENTED	Exact match first, then Gemini picks a zone or `NONE`	`DeliveryService.CheckZoneAsync`	Yes	None

Feature	Status	What / how	Evidence	Visual demo?	Backend work
Price negotiation	IMPLEMENTED (multi-turn confirm NOT VERIFIED live)	Offer ≥ MinPrice → accept (capped at list price) and place the order; within 10% below MinPrice → counter at MinPrice; lower → reject. Confirmation requires Gemini to classify "confirm" as PriceNegotiation.	`NegotiationService.EvaluateOfferAsync`, `HandleNegotiationAsync (:449)`; `MinimumPriceTests`	Yes	B5 preview optional
Reorder	IMPLEMENTED	Re-creates the last non-cancelled order at current prices; skips inactive items	`HandleReorderAsync (:713)`	Yes	None
Fallback handling	IMPLEMENTED	Fallback JSON, `FallbackIntent`, low-confidence results not cached, guaranteed reply	`GeminiService.cs:466`	Yes: "AI unavailable" demo state	None
Parse cache	IMPLEMENTED	10 minutes, keyed by name + message hash + context prefix	`GeminiService.cs:46-63`	Technical page	None

3.4 Security and reliability

Mechanism	Status	Evidence	Tests
JWT bearer on all admin endpoints	IMPLEMENTED	`[Authorize]` on 6 controllers; `Program.cs` validation parameters	`AdminEndpoints_RequireAToken`, `PaymentDecisions_RequireAdminLogin`
Startup refusal on weak admin password or JWT secret	IMPLEMENTED	`Program.cs`	`AppRefusesToStart_WithoutAProperPassword`, `PasswordPolicy`
Constant-time credential check	IMPLEMENTED	`AuthController.SecretEquals`	`WrongOrDefaultCredentials_AreRejected`
Login rate limit (5/min/IP)	IMPLEMENTED	`Program.cs` `"login"` policy	`LoginIsRateLimited_EvenForCorrectPasswordAfterTheLimit`
HMAC-SHA256 webhook signature	IMPLEMENTED	`HmacVerificationMiddleware.cs:18`	`BadSignature_IsRejected_AndDoesNothing`, `HmacVerificationTests`
Webhook global limit (600/min) + per-sender limit (20/min)	IMPLEMENTED	`Program.cs`, `WhatsAppWebhookController.cs:103`	`SenderOverLimit_IsDropped…`, `UnsignedFlood_CannotExhaustWebhookRateLimit`
Idempotency (message ID claimed once)	IMPLEMENTED	`INSERT IGNORE` into `ProcessedWebhookMessages` (`WhatsAppWebhookController.cs:164`)	`SameMessageId_DeliveredTwice…`, `…Concurrently_CreatesOneOrder`
Price floor / list-price cap enforced server-side	IMPLEMENTED	`OrderService.IsAllowedPrice (:406)`	`MinimumPriceTests` (8)
Oversell prevention under concurrency	IMPLEMENTED	Conditional `ExecuteUpdate` + ordered locking	`StockConcurrencyTests` (5)
Order privacy (customers see only their own orders)	IMPLEMENTED	`HandleOrderStatusAsync` scoping; slip matching scoped to the sender	`OrderPrivacyTests` (5)
Slip/reference single use	IMPLEMENTED	Unique indexes + duplicate-key detection	`SameImage_CannotPayASecondOrder…`, `SameSlip_SubmittedConcurrently…`
Owner-only stock increases via WhatsApp	IMPLEMENTED	`DocumentService.IsOwnerPhone` (digits-only compare, fails closed)	`SupplierInvoiceTests` (7)

Mechanism	Status	Evidence	Tests	
Input validation	PARTIALLY IMPLEMENTED	DTO attributes on products; file size and MIME checks; AI quantities and prices re-validated; order DTOs have no attributes but the service validates	`ProductDto.cs`, `DocumentsController.cs`	`InvalidQuantityTests`
Phone masking in logs	IMPLEMENTED	`MaskPhone` helpers	—	
Error handling / isolation	IMPLEMENTED	see 2.8	`FailedItem_RollsBackEarlierItems_AndServiceRemainsUsable`	
Background processing	IMPLEMENTED	`MaintenanceService` (`PeriodicTimer`, 15 min)	`Expiry_*` and `ExpiryThatLosesRace…` (4)	
Blocked customers	NOT IMPLEMENTED	`IsBlocked` never checked	—	
Health check with DB	NOT IMPLEMENTED	`AddHealthChecks()` without checks; the MySQL package is referenced but unused	—	
Hangfire jobs	NOT IMPLEMENTED	Only `.env` keys	—	

3.5 Technical capabilities

Capability	Status	Evidence
Swagger / OpenAPI with JWT security scheme and XML comments	IMPLEMENTED (Development only)	`Program.cs:140-168`
Health endpoint	PARTIALLY IMPLEMENTED (liveness only)	`Program.cs:201`
EF Core migrations + seed data	IMPLEMENTED	4 migrations, `HasData`
Unit tests	IMPLEMENTED (17 methods; NOT VERIFIED run)	`UnitTest1.cs`
Integration tests on real MySQL with faked upstreams	IMPLEMENTED (63 methods; NOT VERIFIED run)	`SecurityAndReliabilityTests.cs`, `TestApp.cs`
CI (build, lint, test)	IMPLEMENTED (NOT VERIFIED green)	`.github/workflows/ci.yml`
Frontend tests	NOT IMPLEMENTED	—
Deployment configuration (Docker, hosting, prod CORS)	NOT IMPLEMENTED	—
Static product images	NOT IMPLEMENTED (files exist in `wwwroot/images/products/`, but there is no `UseStaticFiles`; seed uses Unsplash URLs)	`Program.cs`

3.6 README claims that differ from the code

README claim	Actual
"Models/ Entities (9 tables)"	11 tables
"14 domain services"	13 services in `Services/` (+2 integrations)
Interactive **Track Order / Cancel Order** buttons	Sent, but taps are not handled (G4)
"Tri-lingual support… responds in English, Sinhala, or Tamil"	Partial: several key replies are English-only (G6)
Order Details shows "fraud flags"	Displayed, but never set by any code (G7)
"Gemini retry… up to 3 times"	3 *attempts* in total (initial + 2 retries) for text; Vision retries only on HTTP 429 (60 s wait)

README claim	Actual
Dashboard with "real-time analytics"	Request/response on page load; no push or polling

4. Feature-to-Page Mapping

Feature	Page(s)	Mode	Backend needed
Analytics summary, KPIs, charts	A2 Overview, A13 Analytics	Live + Demo	—
Low-stock alerts	A2 Overview, A3 Products	Live + Demo	—
Product CRUD, bulk tiers, MinPrice	A3 Products, A4 Product detail	Live + Demo (demo edits are local only)	—
Inventory audit trail	A4 Product detail (tab)	Demo; Live after B9	B9 (optional)
Customers + language preference	A7 Customers, A8 Customer detail	Demo; Live after B1	B1, B2
Orders list/detail, cancel, status	A5 Orders, A6 Order detail	Live + Demo	B12 (recommended)
Payment slip review, approve/reject	A11 Payments, A6 (panel)	Live + Demo	—
PDF invoice	A6 (invoice card), R2 Journey	Demo preview; Live after optional invoice endpoint	optional
Delivery transitions + notifications	A12 Delivery (board)	Live + Demo	—
Delivery zones + fuzzy checker	A12 Delivery (Zones tab), A10 Playground	Live + Demo	—
Unpaid expiry	A5/A6 (countdown chip), R2	Live + Demo	—
Supplier invoice OCR → restock	A14 Documents	Live + Demo	B10
WhatsApp conversations	A9 Inbox	**Demo**; Live list after B3; transcript after B4	B3, B4
Intent + language detection	A9 (annotations), A10 Playground, R2	Demo; Live playground after B5	B5
Smart search / visual search	A10 Playground	Live + Demo	—
Negotiation rules	A9 (demo thread), A10 (rule simulator)	Demo; Live preview after B5	B5 (optional)
Webhook security, idempotency, rate limits	R5 Security & Reliability, R3 Architecture	Static	—
Integration status (configured yes/no, DB health)	A15 Integrations	Demo; Live after B6	B6
Tests and CI	R5	Static	—
API surface	R4 API Reference	Static	—

5. The WhatsApp Customer Journey (Traced From Code)

5.1 Step-by-step trace

#	Step	What actually happens	Code	Data passed	Errors / fallback	Status
1	Customer sends a message	Text, image, document or audio in WhatsApp	—	—	—	External
2	Cloud API receives it	Meta posts a webhook to `POST /api/v1/webhook/whatsapp`. The one-time `GET` handshake must already be done (`Verify`).	`WhatsAppWebhookController.Verify`	`hub.*` query	Token mismatch → 401	IMPLEMENTED (live NOT VERIFIED)
3	Signature validation	Middleware buffers the raw body, computes HMAC-SHA256 with `WhatsApp:AppSecret`, compares it to `X-Hub-Signature-256`	`HmacVerificationMiddleware.InvokeAsync (:18)`	raw body, header	Missing or invalid signature or secret → 401, nothing processed	IMPLEMENTED + tested
4	Rate limit + dedupe	Global 600/min fixed window, then per message: per-sender 20/min (excess silently dropped), then `INSERT IGNORE` of the message ID (duplicate → skipped)	`Program.cs` limiter; `Receive (:103, :112)`	`message.From, message.Id`	Duplicate → ignored; failure before claim → HTTP 500 so Meta retries	IMPLEMENTED + tested
5	Acknowledge	Mark as read + typing indicator (best effort)	`MarkAsReadWithTypingAsync (:125)`	wamid	Errors swallowed	IMPLEMENTED
6	Route by type	text → `ProcessTextMessageAsync`; image → payment context? slip pipeline : visual search; document → owner? invoice : open order? slip : refuse; audio → "coming soon"; other (incl. **interactive button replies**) → "couldn't process this type"	`ProcessMessageAsync (:172)`, `HandleImageMessageAsync (:227)`, `HandleDocumentMessageAsync (:278)`	type, media id	Download failure → retry prompt	IMPLEMENTED; button replies NOT IMPLEMENTED
7	Customer + conversation	Get or create `Customer` by phone; get or create `Conversation`; reset to Greeting if idle ≥ 2 h	`GetOrCreateCustomerAsync (:894)`, `GetOrCreateConversationAsync (:918)`	phone, profile name	—	IMPLEMENTED
8	AI language + intent	Builds the catalogue summary (all active products with stock) + structured context, then calls Gemini once for JSON {intent, language, orderItems, productSearchQuery, orderNumber, offeredPrice, replyMessage, confidence}	`BuildCatalogueSummaryAsync`; `GeminiService.ParseMessageAsync (:36)`	message text, name, context, catalogue	3 attempts, then fallback JSON → intent `Other`; JSON repair; cache 10 min (only if confidence > 0.3)	IMPLEMENTED (live NOT VERIFIED)
9	State + language saved	`Customer.PreferredLanguage` updated; `Conversation.State` mapped from intent	`ProcessTextMessageAsync (:72-90)`	—	—	IMPLEMENTED
10	Business operation	One handler per intent (table 5.2). All prices and stock come from the database. AI-supplied prices go through negotiation rules only.	`switch (parsed.Intent) (:94)`	parsed intent	Each handler has clarifying replies; top-level catch sends "Sorry, something went wrong…"	IMPLEMENTED

#	Step	What actually happens	Code	Data passed	Errors / fallback	Status
11	Reply generated	Deterministic templates (si/ta/en where available). Gemini's `replyMessage` is used only for `Other`.	handlers	—	`GetHelpMessage` if the AI reply is empty	PARTIAL (English-only templates, G6)
12	Sent via Cloud API	`POST graph.facebook.com/v22.0/{phoneNumberId}/messages` (text, interactive, image link, or uploaded document)	`WhatsAppSendService.PostToWhatsAppAsync` (`:155`)	JSON payload, bearer token	Non-2xx is **logged, not thrown**	IMPLEMENTED (live NOT VERIFIED)
13	Nothing is persisted as a transcript	No inbound or outbound text is stored	—	—	—	**NOT IMPLEMENTED (G1)**

5.2 Intent handlers (step 10 detail)

Intent	Handler	Business operation	Languages of reply
Greeting	`HandleGreetingAsync` (`:213`)	Menu. The "returning customer" branch is unreachable (G3).	si / ta / en
ProductSearch	`HandleProductSearchAsync` (`:782`)	Gemini top-3 smart search → text list + one image message per product	header/footer si/ta/en; stock lines English
Order	`HandleOrderAsync` (`:559`)	5-strategy name match → quantity check → if an offer is included, route to negotiation → `CreateOrderAsync` → confirmation with bank details + Track/Cancel buttons	errors si/ta/en; **confirmation English only**
Reorder	`HandleReorderAsync` (`:713`)	Last non-cancelled order at current prices	si/ta/en; confirmation English
PriceNegotiation	`HandleNegotiationAsync` (`:449`)	`EvaluateOfferAsync`: accept → order placed immediately; counter → context saved, "reply confirm"; reject	si/ta/en
OrderStatus	`HandleOrderStatusAsync` (`:278`)	Scoped to the sender's own orders only	si/ta/en (status names English)
DeliveryInfo	`HandleDeliveryInfoAsync` (`:347`)	Lists the 10 cheapest active zones. **Does not quote a specific area.** `CheckZoneAsync` is not used in chat even though the footer says "Tell me your area".	si/ta/en
Complaint	`HandleComplaintAsync` (`:399`)	Logs a warning + apology. No ticket is created and the owner is not alerted.	si/ta/en
PaymentConfirmation	`HandlePaymentTextAsync` (`:430`)	Asks for a slip photo	si/ta/en
Other / fallback	default (`:140`)	Gemini reply or help menu	AI reply language

5.3 Image and payment branch

```
Image received --> "Got your image! Analyzing..."  (webhook)
     |
     +-- IsPaymentContextAsync? (state=Ordering OR unpaid Confirmed/Processing order)
     |        yes --> PaymentMatchingService.ProcessPaymentSlipAsync
     |                  Document row saved (audit) -> Gemini Vision OCR
     |                  confidence < 0.5 ............ "send a clearer photo"
     |                  no amount ................... "couldn't extract the amount"
     |                  no own unpaid order ......... "contact us" + owner alert
     |                  amount differs > 1% ......... "mismatch" + owner alert
     |                  slip hash/reference reused .. "already submitted" + owner alert
     |                  OK -> PaymentSlip saved, Order -> PendingReview
     |                        customer: "Payment slip received" ; owner: "needs review"
     |                  ADMIN approves in dashboard -> Paid -> confirmation + PDF invoice
     |
     +-- no --> "Analyzing your image..." (second ack) -> Gemini Vision attributes
                  -> weighted catalogue match -> top 3 text reply
```

The customer receives **two acknowledgements** on the visual-search path (webhook `:239` and `MessageProcessingService.cs:188`). This is minor, but it will be visible in an honest demo.

5.4 Incomplete or unverified parts of the journey

1. No transcript storage (G1), so a real inbox cannot be built yet.
2. Button replies unhandled (G4).
3. No delivery address or area capture in chat (G5), and no area-specific quote in chat (5.2).
4. English-only confirmation, payment, delivery and image replies (G6).
5. Returning-customer greeting unreachable (G3).
6. The negotiation "confirm" step depends on Gemini classifying "confirm" as `PriceNegotiation` while a negotiation context exists (NOT VERIFIED live). The accept path places the order directly and never sends the "Reply *confirm*" text built in `BuildAcceptResult`.
7. Complaints are not escalated to the owner.
8. Everything that calls Meta or Gemini is covered only by faked upstreams in tests. Live behaviour is NOT VERIFIED in this investigation.

5.5 How to demonstrate the journey visually

- **Split-screen "Journey Player" (R2):** on the left, a phone-style WhatsApp chat (fictional). On the right, a vertical pipeline showing the 13 steps above. Each customer message animates through the steps that actually run. Each step card shows the real file and method name, the data passed (sanitised JSON), and a status chip (IMPLEMENTED / PARTIAL).
- **Annotated messages** in the Inbox (A9): every inbound bubble carries chips for *language*, *intent*, *confidence*. Every outbound bubble carries a *source* chip: "Template" or "Gemini reply". Business side effects appear inline as system events, such as "Order ORD-2026-042 created · stock −1" or "Payment → PendingReview".
- **Honesty markers:** a persistent "Simulated conversation · fictional data" banner. English-only replies in a Sinhala or Tamil thread get a small "English template in current build" note. Track/Cancel buttons render as *sent* and are visibly disabled with the tooltip "Reply handling not implemented yet".
- **Live proof without WhatsApp:** the AI Playground (A10) calls the real `POST /products/search`, `POST /products/visual-search` and `POST /delivery-zones/check` endpoints (and `POST /ai/parse` after B5). This shows real Gemini output **without sending any WhatsApp message**.

5.6 Sample conversations (fictional customers, fictional data)

All names, numbers, order numbers, references and bank details below are invented for the demo. Product names and prices come from the **seed catalogue** in `AppDbContext.cs`, which is sample data and not business records. Bot texts are copied from the actual templates in the code. Where the code replies in English inside a Sinhala or Tamil thread, the sample shows that exactly as it happens.

Conversation A: Sinhala / Singlish ordering (demo customer "Kasun D." · +94 70 000 0101)

From	Message	Annotation
Customer	ආයුබෝවන්	lang `si` · intent Greeting
Bot	🙏 ආයුබෝවන්! SellBot.lk වෙත සාදරයෙන් පිළිගනිමු. අපි ශ්‍රී ලංකාවේ ෆර්නිචර් ව්‍යාපාරයකි. ඔබට WhatsApp හරහා ඇණවුම් කළ හැකිය! 📦 *browse* - භාණ්ඩ බලන්න · 🛒 ඔබට අවශ්‍ය දේ ලියන්න · 📸 ෆොටෝ එකක් යවන්න - සමාන භාණ්ඩ හොයනවා	Template (si) · new-customer branch
Customer	සෝෆා තියෙනවද?	lang `si` · intent ProductSearch
Bot	🛋️ *1 ගැළපෙන භාණ්ඩ:* · *Three-Seater Fabric Sofa* 💰 LKR 55,000 ✅ In stock (12 available) 🤌 5% off for 2+ units · ඔඩරය දෙන්න නමෙන් ලියන්න, නැතිනම් තවත් විස්තර ඉල්ලන්න!	Gemini smart search (≤3 results) + image message. Stock line is English in code.
Customer	chair ekak denna	Singlish · lang `si` · intent Order · item "Executive High-Back Leather Chair" ×1
Bot	✅ *Order Confirmed!* 📋 Order Number: *ORD-2026-041* · Executive High-Back Leather Chair x1 — LKR 28,500 · 💰 *Total: LKR 28,500* · 🏧 Transfer to: Bank *Demo Bank* · Account *000-000000* · Name *SellBot Demo* · Send a screenshot of your payment slip here · ⏰ We hold your items for 24 hours... [Track Order] [Cancel Order]	Order created · stock 20→19 · **English-only template** · buttons sent (taps not handled)
Customer	(sends slip image)	Payment context: unpaid Confirmed order
Bot	📸 Got your image! Analyzing it now, please wait... → 📄 Payment slip received, thank you! Order: ORD-2026-041 · Amount: LKR 28,500 · Reference: DEMO123456 · Our team will verify your payment and confirm shortly. 🙏	OCR confidence 0.97 · Order → **PendingReview** · owner alerted
System	Admin approves on the dashboard	Order → **Paid**
Bot	✅ Payment confirmed! Order: ORD-2026-041 ... Thank you! We'll start processing your order now. 🙏 + 📄 *SellBotLk-ORD-2026-041.pdf*	QuestPDF invoice sent as a document

Conversation B: Tamil support (demo customer "Priya S." · +94 70 000 0102)

From	Message	Annotation
Customer	வணக்கம்	lang `ta` · Greeting
Bot	🙏 வணக்கம்! SellBot.lk-க்கு வரவேற்கிறோம். நாங்கள் இலங்கையில் தளபாடங்கள் விற்கிறோம். WhatsApp மூலம் ஆர்டர் செய்யலாம்! 📦 *browse* - பொருட்களை பார்க்க · 🛒 உங்கள் தேவையை சொல்லுங்கள் · 📸 படம் அனுப்பி பொருள் தேடுங்கள்	Template (ta)
Customer	யாழ்ப்பாணத்துக்கு டெலிவரி கட்டணம் என்ன?	lang `ta` · DeliveryInfo
Bot	🚚 *டெலிவரி பகுதிகள் மற்றும் கட்டணங்கள்:* 📍 *Colombo* — LKR 300 | 1 day(s) (Free over LKR 50,000) · 📍 *Gampaha* — LKR 400 | 1 day(s) ... _...and 15 more zones. Ask for a specific area!_ · உங்கள் பகுதியை சொல்லுங்கள் — குறிப்பிட்ட கட்டணம் சொல்கிறோம்!	Lists the 10 cheapest zones. **Jaffna (LKR 1,500) is not quoted specifically**, which is a known gap.
Customer	என் ஆர்டர் நிலை என்ன?	lang `ta` · OrderStatus (no number → latest own order)
Bot	🚚 *உங்கள் ஆர்டர் — ORD-2026-037* · நிலை: *Dispatched* · கட்டணம்: ✅ Paid · மொத்தம்: LKR 18,500 · ஏதேனும் கேள்வி இருந்தால் தொடர்பு கொள்ளுங்கள்.	Scoped to the sender's own orders (`OrderPrivacyTests`)

Conversation C: English negotiation (demo customer "Ayesha F." · +94 70 000 0103)

From	Message	Annotation
Customer	Can I get the fabric sofa for 42,000?	lang `en` · PriceNegotiation · offer 42,000
Bot	🤝 Best we can do for Three-Seater Fabric Sofa is LKR 45,000 each — that's our lowest price. 💰 Total for x1: LKR 45,000 · Reply *confirm* to accept this price.	Rule: MinPrice 45,000; 42,000 ≥ 40,500 (10% band) → **counter**; context saved to `Conversation.Context`
Customer	confirm	intent PriceNegotiation (expected) + active context
Bot	✅ *Order Confirmed!* … Three-Seater Fabric Sofa x1 — LKR 45,000 … [Track Order] [Cancel Order]	`NegotiatedPrice` 45,000 re-validated by `IsAllowedPrice`
Customer	Bookshelf for 11,000?	offer 11,000 < 12,600 (MinPrice 14,000 × 0.9)
Bot	😔 Sorry, LKR 18,500 is our standard price for Wooden Bookshelf (5-tier) and we're unable to go lower. Would you like to order at the regular price?	**Reject** path

Rule notes for the demo script: an offer **at or above** MinPrice is *accepted* and the order is placed immediately at `min(offer, list price)`, so an offer above list price is charged at list price, never more. An offer in the 10% band below MinPrice gets a counter at MinPrice. Anything lower is rejected. Demo fixtures must be produced from these rules (Section 7.4).

6. Recommended Website Structure

6.1 Three areas, two layouts

Area	Purpose	Layout	Pages
Admin Dashboard	Business management (real where implemented)	**Admin layout**: sidebar + top bar + content	A1–A8, A11–A15
WhatsApp AI Experience	How customers interact; AI transparency	Admin layout for the Inbox and Playground; **Presentation layout** for the Journey Player	A9, A10, R2
Technical Showcase	Engineering story for recruiters	**Presentation layout**: top navigation, wide reading column, section anchors	R1, R3, R4, R5

6.2 Route map

```
/                       R1 Recruiter landing       (demo build) | redirect -> /app (live build)
/login                  A1 Login                   (live build only)
/app                    A2 Overview
/app/orders             A5 Orders          /app/orders/:id        A6 Order detail
/app/payments           A11 Payment review
/app/delivery           A12 Delivery (tabs: Board | Zones)
/app/products           A3 Products        /app/products/:id      A4 Product detail
/app/customers          A7 Customers       /app/customers/:id     A8 Customer detail
/app/inbox              A9 WhatsApp inbox  /app/inbox/:id         (mobile: conversation)
/app/ai-playground      A10 AI playground
/app/analytics          A13 Analytics
/app/documents          A14 Documents & OCR
/app/integrations       A15 Integrations & status
/showcase/journey       R2 Customer journey player
/showcase/architecture  R3 Architecture & stack
/showcase/api           R4 API reference
/showcase/security      R5 Security, reliability & testing
```

6.3 Sidebar (admin layout)

```
[Logo] SellBot.lk                      [DEMO | LIVE pill]
-----------------------------------------------
Overview
COMMERCE      Orders · Payments (n pending) · Delivery · Products · Customers
WHATSAPP & AI Inbox · AI Playground
INSIGHTS      Analytics · Documents
SYSTEM        Integrations
-----------------------------------------------
(demo build)  About this project -> Journey · Architecture · API · Security
(live build)  Signed in as <admin> · Log out
```

The order reflects daily work. Orders and Payments sit at the top because the payment-review queue is the owner's main manual task in the code. Badge counts come from data already available: `PendingReview` orders and low-stock products.

6.4 What recruiters should see on the technical side

Show: the architecture diagram, the AI guard-rail principle ("AI classifies; code decides prices, stock, ownership and payment"), webhook security layers, idempotency and concurrency guarantees **mapped to named tests**, the state machines (Section 2.4), the stack with versions, the curated endpoint list, and the trade-offs and known limitations (Section 14). Honest limitations read as seniority.

Do **not** show: secrets, real phone numbers, real slips, production URLs, raw Swagger from a live server, stack traces, or claims beyond Section 3.

7. Live Admin Mode and Recruiter Demo Mode

7.1 Recommendation

Use **one codebase with two build targets**, selected at build time with `VITE_DATA_MODE=live|demo`:

	Live Admin Mode	Recruiter Demo Mode
Audience	The business owner	Recruiters, interviewers, public
Data	Real API (`VITE_API_BASE_URL`)	Bundled fictional fixtures, in-memory
Auth	`/login` → JWT	None. "Enter demo" button on the landing page.
Mutations	Real (approve payment sends a real WhatsApp message and invoice)	Simulated, reset on reload, toast "Simulated — nothing was sent"
Hosting	Private (localhost or a restricted host next to the API)	Public static hosting
Pages	A1–A15 (A9 limited until B3/B4)	R1–R5 + A2–A15
Banner	"LIVE · actions affect real customers"	"DEMO · fictional data · no real messages"

Why a build-time switch instead of a runtime toggle: the public demo bundle then contains no API base URL and no live client code, so a recruiter (or attacker) cannot point the public site at production. A runtime toggle would ship both paths and invite mistakes.

7.2 Data-source adapter (frontend architecture)

```
src/data/
  source.ts        interface DataSource { listOrders(q), getOrder(id), approvePayment(id),
                   rejectPayment(id, reason), updateDeliveryStatus(...), listProducts(),
                   listCustomers(q), listConversations(), getConversation(id),
                   aiParse(text), smartSearch(q), visualSearch(file), checkDelivery(...),
                   getAnalytics(from,to), listDocuments(...), getSystemStatus() ... }
  live.ts          implements DataSource with the existing sellbotApi + new B-endpoints
  demo/            implements DataSource from fixtures; simulated latency + errors
    fixtures/      products.ts (seed catalogue), customers.ts, orders.ts,
                   conversations.ts, documents.ts, analytics.ts (derived from orders)
    rules.ts       TypeScript mirror of the negotiation, bulk-discount and expiry rules
  index.ts         export const data: DataSource = import.meta.env.VITE_DATA_MODE === 'demo'
                     ? demoSource : liveSource      // dead branch removed at build time
```

Pages call only `data.*`. They never call `fetch` directly. Methods with no backend yet (for example `listCustomers` before B1) throw `NotAvailableInLive` in `live.ts`, and the page shows an "Available in demo · backend endpoint planned" empty state. This avoids faking live data.

7.3 Privacy and safety rules

1. **Fixtures are hand-written.** Never export or copy production rows, screenshots, slips or chat logs into the repo.
2. Fictional phone numbers use one obviously fake range (`+94 70 000 01xx`) and are **displayed masked** in both modes (`+94 70 ••• ••01`). Live mode gets a per-row "reveal" button.
3. Demo payment slips are generated SVGs watermarked "SAMPLE — NOT A REAL TRANSFER". Demo bank details are "Demo Bank / 000-000000".
4. Live slip images load only on an explicit click (already the behaviour in `OrderDetailsPage`) and are never cached by a service worker.
5. Integration status (B6) returns **booleans only**, never tokens, IDs or secrets. The UI never renders configuration values.

6. `GET /documents` returns raw `ExtractedData`, which can contain sender and receiver names from slips. B10 should return a DTO, and the UI must mask names.
7. Swagger stays Development-only. R4 is a curated static page, not a live Swagger proxy.
8. No analytics or tracking scripts on Live builds. The demo can use privacy-friendly page counts if wanted.
9. A pre-deploy check: `grep` the demo `dist/` for `localhost:5028`, `api/v1`, and the production host before publishing (Phase 5 checklist).

7.4 Keeping demo data truthful

- Generate demo analytics **from demo orders** (same aggregation as `AnalyticsService`), so charts and tables agree.
- Implement the negotiation, bulk-discount and expiry rules in `demo/rules.ts` from the C# code, with unit tests using the same values as `MinimumPriceTests`. The Conversation C mistake in 5.6 is exactly what this prevents.
- Demo conversations must only contain bot replies that exist as templates in the code. Mark gaps (English-only replies, unhandled buttons).

8. Page Specifications

Priority: **P0** = MVP, **P1** = strongly recommended, **P2** = optional. Mode: **L** = live, **D** = demo.

A1 Login

Field	Specification
Why	Gate for Live mode; already exists (`LoginPage.tsx`)
Demonstrates	JWT auth, rate-limited login (error message for HTTP 429)
Users	Business owner
Layout	Centred card on a branded background (Presentation-like, no sidebar)
Sections	Logo, username, password, submit, error area, "Live environment" notice
Components	AuthCard, TextField, PasswordField (show/hide), Button, InlineAlert
Data / endpoints	`POST /auth/login`
Missing backend	None. Optional: return `expiresAt` so the UI can warn before the 24-hour expiry.
Mode / Priority / Phase	L only · P0 · Phase 1

A2 Overview (dashboard home)

Field	Specification
Why	First screen. Answers "what needs my attention?" Different from Analytics, which answers "how is the business doing?"
Demonstrates	Real KPIs + action queues driven by the payment review and stock rules
Users	Owner; recruiter (demo)
Layout	KPI row (4) → two-column: "Needs action" (payments pending review, unpaid orders about to expire, low stock) + "Recent orders" → mini revenue sparkline
Components	StatCard, ActionQueueCard, OrderStatusBadge, PaymentStatusBadge, MiniAreaChart, DataTable (compact), EmptyState
Data / endpoints	`GET /analytics/summary` (last 30 days), `GET /orders` (filter `paymentStatus=PendingReview` client-side), `GET /products/low-stock`
Missing backend	None (B8 would make the pending-review count cheaper)
Mode / Priority / Phase	L + D · P0 · Phase 2

A3 Products

Field	Specification
Why	The API supports full CRUD but there is **no UI today**. The catalogue drives every AI answer.
Demonstrates	Catalogue with MinPrice (negotiation floor), bulk tiers, stock health, attributes used by visual search
Users	Owner
Layout	Toolbar (search, category filter, stock filter, "Add product") → table or grid toggle → row click opens A4

Field	Specification
Components	DataTable, ProductThumb, StockMeter, CategoryChip, SearchInput, FilterSelect, ViewToggle, Drawer, ProductForm, ConfirmDialog
Data / endpoints	`GET /products`, `GET /products/low-stock`, `POST/PUT/DELETE /products`
Missing backend	None. Search, filter and sort are done client-side (the catalogue is small). Note that `GET /products` returns active products only, so deactivated items cannot be listed or restored from the UI (optional `?includeInactive`).
Mode / Priority / Phase	L + D · P0 · Phase 2

A4 Product detail

Field	Specification
Why	Shows the pricing rules that make negotiation safe
Demonstrates	Price vs MinPrice vs bulk price ("lowest the bot will ever accept"), stock, AI attributes, image
Layout	Desktop: right-side drawer over A3 with a deep link `/app/products/:id`. Mobile: full page. Tabs: Details · Pricing rules · Inventory history.
Components	Drawer, Tabs, KeyValueList, PriceLadder (list → bulk → floor), StockMeter, ProductForm, Timeline (inventory)
Data / endpoints	`GET /products/{id}`, `PUT /products/{id}`, `DELETE /products/{id}`
Missing backend	B9 for the Inventory history tab (demo uses fixtures)
Mode / Priority / Phase	L + D · P1 · Phase 2

A5 Orders

Field	Specification
Why	Exists today (`OrdersPage.tsx`); redesign it into the main work list
Demonstrates	Order + payment state, source channel (WhatsApp vs admin), negotiated price marker, expiry countdown for unpaid orders
Layout	Status tabs (All, Confirmed, Processing, Dispatched, Delivered, Cancelled) + payment filter chips (Unpaid, Pending review, Paid) + search (order no. / customer) → table → row → A6
Components	Tabs, FilterChips, SearchInput, DataTable (sortable, client pagination), OrderStatusBadge, PaymentStatusBadge, CountdownChip, MaskedPhone
Data / endpoints	`GET /orders?status&customerId`
Missing backend	B8 (optional: server pagination, payment filter, search) once order volume grows
Mode / Priority / Phase	L + D · P0 · Phase 2

A6 Order detail

Field	Specification
Why	Exists today; central place for actions
Demonstrates	Line items (unit vs negotiated price), discount, two state machines (fulfilment + payment), payment review, cancellation with stock restore
Layout	Header (number, badges, primary actions) → left: Items table, Totals, Customer card (masked) · right: Payment card (slip preview, approve/reject), Fulfilment stepper, Activity timeline

Field	Specification
Components	PageHeader, StatusStepper, PaymentReviewPanel (shared with A11), SlipViewer, ItemsTable, TotalsCard, CustomerMiniCard, ConfirmDialog (with "this will WhatsApp the customer" warning in Live), Timeline
Data / endpoints	`GET /orders/{id}`, `GET /orders/{id}/payment-slip`, `PUT …/payment/approve`, `PUT …/payment/reject`, `PUT …/delivery-status`, `PUT …/cancel`
Missing backend	B12: use `delivery-status` for all fulfilment moves in the UI, and drop the generic status select. Hide the fraud badge (never set). The activity timeline in Live is limited to CreatedAt/UpdatedAt (no event log); demo shows a full timeline labelled "illustrative".
Mode / Priority / Phase	L + D · P0 · Phase 2 (payment panel redesigned in Phase 4)

A7 Customers

Field	Specification
Why	Customers exist in the database but are invisible in the UI
Demonstrates	Auto-created WhatsApp customers, preferred language (si/ta/en), order history value
Layout	Search + language filter → table (name, masked phone, language flag, orders, spend, last order)
Components	DataTable, LanguageChip, MaskedPhone, SearchInput, EmptyState
Data / endpoints	None today (only names via orders)
Missing backend	**B1** (`GET /customers`, computed aggregates), **B2** (fix counters)
Mode / Priority / Phase	D (L after B1) · P1 · Phase 2

A8 Customer detail

Field	Specification
Why	Ties a customer to orders and their conversation state
Layout	Profile header (masked phone, language, since) → KPIs (orders, spend) → Orders table → Conversation state card → link to Inbox thread (demo)
Components	ProfileHeader, StatCard, DataTable, ConversationStateBadge
Data / endpoints	`GET /orders?customerId=` exists; profile needs **B1**; conversation state needs **B3**
Mode / Priority / Phase	D (L after B1) · P2 · Phase 2

A9 WhatsApp Inbox (with conversation pane)

Field	Specification
Why	The core of the product story: customers buy through chat
Demonstrates	Multilingual threads, intent/language/confidence per message, template vs AI replies, business side effects inline, negotiation context, image/slip routing
Layout	Three panes on desktop: conversation list (search, language and state filters) · chat thread (bubbles, system events, "typing…") · context panel (customer, conversation state, active negotiation, open order, AI trace for the selected message). Tablet: two panes. Mobile: list → thread route.
Components	ConversationList, ConversationListItem (unread dot, state badge, language chip), ChatThread, MessageBubble (in/out, text/image/document/interactive), SystemEventRow, IntentChip, ConfidenceMeter, AITracePanel, NegotiationCard, DemoComposer (pick a canned customer message), SimulationBanner

Field	Specification
Data / endpoints	**None for transcripts** (G1). Live after B3: conversation list with state + last activity only. Live after B4: full transcripts.
Missing backend	**B3**, **B4** (optional, privacy review needed)
Mode / Priority / Phase	**D** (L partial after B3/B4) · P0 (demo) · Phase 3

A10 AI Playground

Field	Specification
Why	Proves the AI is real without touching WhatsApp
Demonstrates	Intent + language parse, smart search, visual search, fuzzy zone match, negotiation rules
Layout	Tabs or cards: "Understand a message" (text in → JSON + chips) · "Find products" (query → ranked products + WhatsApp-formatted preview) · "Search by photo" (upload → detected attributes + scores) · "Delivery area" (free text → matched zone, fee) · "Negotiation rules" (product + offer → accept/counter/reject explanation)
Components	PlaygroundCard, TextArea, ImageDropzone, JsonViewer (collapsible), ResultCard, ScoreBar, WhatsAppPreviewBubble, ModeNotice ("Live call to Gemini, nothing is sent to WhatsApp")
Data / endpoints	`POST /products/search`, `POST /products/visual-search`, `POST /delivery-zones/check` (all exist); **B5** `POST /ai/parse` (+ optional negotiation preview)
Missing backend	B5 for the parse and negotiation cards in Live
Mode / Priority / Phase	L + D · P1 · Phase 3

A11 Payment review

Field	Specification
Why	The owner's main manual task. Today it is reachable only order by order.
Demonstrates	Human-in-the-loop AI: OCR result vs order total, single-use slips, approve → invoice
Layout	Queue list (PendingReview orders, oldest first) · review pane: slip viewer (zoom) beside an extracted-vs-expected comparison (amount, reference, date) and approve/reject with reason
Components	QueueList, SlipViewer, ComparisonTable (match/mismatch icons), PaymentReviewPanel, RejectReasonDialog, Toast
Data / endpoints	`GET /orders` (client filter `paymentStatus=PendingReview`), `GET /orders/{id}/payment-slip`, approve/reject
Missing backend	The extracted OCR fields (bank, date, names) are in `Documents.ExtractedData` and not linked to the order. The reference and amount are available (`PaymentReference`, `TotalAmount`). Optional: include the slip's extraction in the order DTO.
Mode / Priority / Phase	L + D · P0 · Phase 4

A12 Delivery (Board + Zones)

Field	Specification
Why	Fulfilment transitions send real WhatsApp notifications. Zones are an existing page.
Demonstrates	Validated transitions (Confirmed → Processing → Dispatched → Delivered), driver notes, customer notification, 25 zones with free-delivery thresholds, Gemini fuzzy area match

Field	Specification
Layout	Tab 1 Board: 4 columns, cards with order no., items, area (or "No area captured"), paid status; "Advance" action with note. Tab 2 Zones: zone table + fee checker (existing). Optional Sri Lanka district map (static SVG; no map library needed).
Components	KanbanColumn, DeliveryCard, AdvanceStatusDialog (driver note, "customer will be notified"), ZoneTable, FeeChecker
Data / endpoints	`GET /orders`, `PUT /orders/{id}/delivery-status`, `GET /delivery-zones`, `POST /delivery-zones/check`
Missing backend	None. Drag-and-drop must only allow valid next steps.
Mode / Priority / Phase	L + D · P0 (board) · Phase 4

A13 Analytics

Field	Specification
Why	The existing Dashboard is really an analytics page
Demonstrates	Revenue over time, top products, status distribution, paid vs unpaid, date-range filtering
Layout	Date-range bar → KPI row → revenue area chart (full width) → top products bar + status donut → recent orders
Components	DateRangePicker (native date inputs are fine), StatCard, AreaChart, BarChart, DonutChart, ChartCard (title, legend, empty state)
Data / endpoints	`GET /analytics/summary?from&to`
Missing backend	None. Language mix and intent mix charts are possible only after B4 (do not fake them in Live).
Mode / Priority / Phase	L + D · P1 · Phase 4

A14 Documents & OCR

Field	Specification
Why	Implemented OCR features (supplier invoices → restock, damage reports) have **no UI**
Demonstrates	Gemini Vision extraction → structured table → inventory change with audit
Layout	Upload card (type select: Supplier invoice / Damage report; slips belong in A11) → documents table (type, vendor, total, processed, notes) → detail drawer: extracted JSON as a table, processing notes
Components	Dropzone, DocTypeSelect, DataTable, Drawer, ExtractedTable, JsonViewer, StatusPill
Data / endpoints	`GET /documents`, `GET /documents/{id}`, `POST /documents/process`
Missing backend	**B10** (DTO + masking). Upload in Live **changes stock**, so it needs a confirm dialog.
Mode / Priority / Phase	L + D · P2 · Phase 4

A15 Integrations & System status

Field	Specification
Why	Shows the integration surface (Meta, Gemini, MySQL, background jobs) and operational health
Demonstrates	Configured or not, database reachable, maintenance job cadence, webhook security settings, rate limits
Layout	Integration cards (WhatsApp Cloud API, Gemini, MySQL, Maintenance job) with status dot + "what it is used for" + key facts (Graph v22.0, model name, 15-minute interval)

Field	Specification
Components	IntegrationCard, StatusDot, KeyValueList, InfoTooltip
Data / endpoints	`GET /health` (liveness only)
Missing backend	**B6** (`GET /system/status`, booleans only, plus a DB health check)
Mode / Priority / Phase	D (L after B6) · P2 · Phase 4

R1 Recruiter landing / demo entry

Field	Specification
Why	A recruiter must understand the project in about 30 seconds
Layout	Hero (one-line pitch + "Enter demo" + "View on GitHub") → "How it works" 3-step strip (Customer chats → AI understands → Owner manages) → 3 entry tiles (Admin dashboard · WhatsApp AI · Engineering) → stack badges → honesty note ("Fictional data. Real system architecture.")
Components	Hero, StepStrip, EntryTile, TechBadge, Footer, DemoNotice
Data	Static
Mode / Priority / Phase	D · P0 · Phase 5 (designed in Phase 1)

R2 Customer Journey Player

Field	Specification
Why	Explains the full pipeline visually (Section 5.5)
Layout	Scenario picker (A Sinhala order · B Tamil support · C English negotiation · D photo search) → split view: phone chat (left) + pipeline steps (right) → playback controls
Components	PhoneFrame, ChatThread (shared with A9), PipelineStep (file/method, data, status chip), PlaybackControls, ScenarioTabs
Data	Demo fixtures only
Mode / Priority / Phase	D · P0 · Phase 3

R3 Architecture & stack

Field	Specification
Layout	System diagram (interactive hotspots) → request pipeline → data model → state machines → stack table with versions → design decisions (AI classifies, code decides; DB-enforced idempotency; ordered locks)
Components	DiagramCanvas (static SVG with hotspots), SectionNav (sticky anchors), StateMachineDiagram, TechTable, CalloutCard
Mode / Priority / Phase	Static · P0 · Phase 5

R4 API reference

Field	Specification
Layout	Grouped endpoint list (Auth, Orders, Payments, Products, AI, Documents, Delivery, Webhook) with method badge, auth badge, one-line purpose, sample (fictional) request and response

Field	Specification
Components	EndpointRow, MethodBadge, AuthBadge, CodeBlock, CopyButton
Mode / Priority / Phase	Static · P2 · Phase 5

R5 Security, reliability & testing

Field	Specification
Layout	Threat → mechanism → proof table (Section 3.4) · webhook defence diagram (signature → rate limit → dedupe → per-message isolation) · test suite overview (classes, counts, "runs on real MySQL") · CI badge · known limitations
Components	GuaranteeCard (mechanism + test names), LayeredDefenceDiagram, TestSuiteTable, LimitationList
Mode / Priority / Phase	Static · P0 · Phase 5

8.1 Pages that should NOT be created

Page	Reason
Public storefront, cart, checkout, customer login	Contradicts the product. Customers buy through WhatsApp.
Settings (bank details, tokens, owner phone, model)	Configuration lives in environment variables with no API. Exposing secrets in a UI is a security risk. A15 shows only booleans.
User and role management	One admin from configuration. No user table.
Agent reply / compose box in the Live inbox	No backend. Would send real messages and needs the WhatsApp 24-hour window and templates. Keep it out of scope.
WhatsApp template manager	Templates are not used by the code
Fraud center	`IsFraudFlagged` / `FraudPending` are never set
Daily reports	`DailyReports` is never written
Delivery zone CRUD	No endpoints. Zones are seed data.
Hangfire / jobs dashboard	Hangfire is not installed. A15 covers the one background service.
Voice-note transcription demo	Not implemented (placeholder reply)
Separate tech-stack page and testing page	Merged into R3 and R5 to keep the tour short
Live Swagger embed	Swagger is Development-only by design. Use R4 instead.

9. The Five-Phase Implementation Roadmap

Estimates are **focused developer days for one developer** familiar with the codebase. Figma design time is listed separately for each phase. Calendar time depends on availability.

9.0 Phase dependencies

```
PHASE 1  Foundation, architecture, design system
   |  (layouts, tokens, components, DataSource adapter, demo fixtures skeleton)
   v
PHASE 2  Core business administration  ----------------------+
   |  (Overview, Products, Orders, Order detail, Customers)  |
   v                                                         |
PHASE 3  WhatsApp inbox & AI experience                      |  Phase 4 needs Phase 2's
   |  (Inbox demo, Journey player, AI playground)            |  order/table components,
   v                                                         |  not Phase 3. Phases 3 and 4
PHASE 4  Business workflows & analytics  <-------------------+  can swap order or overlap.
   |  (Payments queue, Delivery board, Analytics, Documents, Integrations)
   v
PHASE 5  Recruiter showcase & production polish
      (Landing, Architecture, API, Security, a11y, perf, deploy demo build)
```

- Phase 1 blocks everything: no page work starts before tokens, layouts and the `DataSource` interface exist.
- Phase 3's chat components (`ChatThread`, `MessageBubble`) are reused by R2 in Phase 5.
- Backend items can be done in parallel with design: B1 and B2 before Phase 2 ends, B3, B5 and B7 before Phase 3 ends, B6 and B10 before Phase 4 ends, B11 before the Phase 5 deployment.

9.1 MVP scope vs optional enhancements

MVP (recruiter-ready, about 20–26 days)	Optional enhancements
P1 complete	A4 inventory history (B9), A8 Customer detail
A2 Overview, A3 Products, A5 Orders, A6 Order detail	A7 Customers in **Live** (B1)
A7 Customers (demo fixtures)	A9 Live inbox (B3 + B4)
A9 Inbox (**demo**), R2 Journey player	A10 Playground Live parse (B5)
A11 Payment review, A12 Delivery board	A13 separate Analytics page, A14 Documents, A15 Integrations (B6)
R1 Landing, R3 Architecture, R5 Security	R4 API reference, frontend test suite, district map
Demo build deployed publicly; Live build runs locally	B7 button replies, B13 delivery capture

---

PHASE 1 — Foundation, Architecture and Design System

#	Item	Detail
1	Objective	Turn `sellbotlk-admin` into a two-layout, two-mode application with a design system, without changing any business behaviour
2	Expected outcome	Sidebar admin shell + presentation shell, route groups, design tokens, core components, `DataSource` interface with `live` (wrapping the current `sellbotApi`) and `demo` (skeleton fixtures), existing 5 pages running inside the new shell
3	Pages included	A1 Login (restyle), app shell for all A-pages, presentation shell for R-pages, the existing Dashboard/Orders/Order details/Zones moved under `/app/*` unchanged

#	Item	Detail
4	Main UI sections	Sidebar (grouped nav, mode pill, badges), top bar (page title, search slot, user/demo menu), mode banner, content area with page header, presentation top nav + footer
5	Components	Tokens (colour roles, type scale, spacing, radius, elevation, z-index, breakpoints); Button, IconButton, Input, Select, SearchInput, Field, Badge (+ OrderStatus/PaymentStatus variants), Card, StatCard, DataTable v1, Tabs, Drawer, Modal/ConfirmDialog, Toast, InlineAlert, EmptyState, Skeleton, PageHeader, ModeBanner, MaskedPhone
6	Backend APIs reused	`POST /auth/login` + everything the current pages already call (no new calls)
7	Frontend tasks	(a) Route groups `/`, `/login`, `/app/*`, `/showcase/*`; keep the `RequireAuth` logic for Live only. (b) `AdminLayout` + `PresentationLayout`. (c) `src/data` adapter (Section 7.2), with every existing page switched from `sellbotApi` to `data.*`. (d) `VITE_DATA_MODE` handling + build scripts `build:live`, `build:demo`. (e) CSS custom-property tokens from Figma; migrate `index.css`; delete the unused `App.css` and template assets; set `<title>`. (f) Shared `useAsync` hook for loading, error and data. (g) Handle 401 / expired token (already partly done) and 429 on login. (h) Responsive shell: sidebar → icon rail (tablet) → drawer (mobile).
8	Backend changes	None required. Optional: return `expiresAt` from login.
9	Dependencies	Figma foundations + shell frames approved first
10	Pinterest keywords	"SaaS admin dashboard sidebar navigation", "dashboard design system components", "admin panel collapsible sidebar", "dashboard UI kit tables badges", "minimal login page SaaS", "status badge design system", "dashboard empty state illustration"
11	Figma requirements	File structure: Cover · Foundations · Components · Templates · Pages (by phase) · Prototype · Handoff. Foundations: colour **roles** (bg, surface, text, muted, border, brand, success, warning, danger, info, + 6 order-status and 4 payment-status roles), type scale, 4/8 px spacing, radius, elevation, grid per breakpoint. Components with variants and states (default/hover/focus/disabled/loading). Templates: AdminLayout (desktop 1440, tablet 834, mobile 390), PresentationLayout (same breakpoints), Login.
12	Acceptance criteria	All 5 existing features still work in Live against the real API. `npm run build` + `npm run lint` pass in CI. The demo build renders the shell and existing pages from fixtures with **zero** requests to any API host. Every colour and spacing value comes from a token. Keyboard focus is visible on every component.
13	Testing	Manual regression of login → dashboard → order → approve/reject (on a test DB, never production). Network-tab check that the demo build makes no API calls. Lint and type-check in CI. Optional: Vitest + Testing Library for `DataSource` selection and `useAsync`.
14	Risks	Refactor breaks existing flows (mitigation: move pages first, restyle later). Over-building the component library (mitigation: build only what Phase 2 needs). Tokens not settled because the visual direction is open.
15	Estimate	5–7 dev days · Figma 3–5 days
16	Completion checklist	☐ tokens in code ☐ two layouts ☐ route groups ☐ DataSource live + demo ☐ existing pages migrated ☐ demo build has no API URL ☐ leftovers removed ☐ CI green
17	Before moving on	Shell approved at 3 breakpoints. `DataSource` interface reviewed against Section 7.2 (adding methods later is fine; changing shapes is costly). Demo fixtures for products and orders exist.

Figma first (Phase 1): Foundations page, component sheet, AdminLayout (3 breakpoints), PresentationLayout (3 breakpoints), Login, and one "reference page" (A5 Orders) to test the system on a real table.

PHASE 2 — Core Business Administration

#	Item	Detail
1	Objective	Present the verified business core (catalogue, orders, customers) with real data in Live and rich fixtures in Demo
2	Expected outcome	A2 Overview, A3 Products, A4 Product detail, A5 Orders, A6 Order detail, A7 Customers, A8 Customer detail (optional)
3	Pages	A2, A3, A4, A5, A6, A7, (A8)

#	Item	Detail
4	Main UI sections	KPI row, needs-action queues, filter toolbars, sortable tables, product drawer with a pricing ladder, order header with dual state badges, fulfilment stepper, payment card (existing logic, restyled), customer table with language chips
5	Components	DataTable v2 (sort, client pagination, row actions, sticky header, responsive card mode), FilterChips, StatusStepper, PriceLadder, StockMeter, ProductForm, ItemsTable, TotalsCard, CustomerMiniCard, LanguageChip, CountdownChip, ActionQueueCard, MiniAreaChart, Timeline
6	Backend APIs reused	`GET /analytics/summary`, `GET/POST/PUT/DELETE /products`, `GET /products/low-stock`, `GET /orders`, `GET /orders/{id}`, `PUT /orders/{id}/delivery-status`, `PUT /orders/{id}/cancel`, `GET /orders/{id}/payment-slip`, payment approve/reject
7	Frontend tasks	Build the pages. Client-side search, filter, sort and pagination. Masked phone everywhere. Confirm dialogs that say "This sends a WhatsApp message to the customer" (Live) or "Simulated" (Demo). Expiry countdown from `UpdatedAt` + 24 h (read the configured hours from B6 later; default 24). Remove the generic status select and keep validated fulfilment actions only (B12). Hide the fraud badge. Demo fixtures: about 40 orders across all states, 12 customers (si/ta/en mix), 5 seed products + 7 fictional extra products.
8	Backend changes	**B1** customers read API (computed aggregates). **B2** maintain or compute customer counters. **B12** align or retire `PUT /orders/{id}/status`. Optional **B8** (orders paging, search, payment filter), **B9** (inventory log read).
9	Dependencies	Phase 1 complete; B1 for Live customers
10	Pinterest keywords	"e-commerce admin dashboard overview", "order management dashboard table filters", "order details page admin timeline", "inventory management product table thumbnails", "product edit side drawer", "CRM customer list table", "customer profile dashboard", "pricing tiers card UI"
11	Figma requirements	Desktop/tablet/mobile frames for each page. States: loading skeleton, empty (no orders yet / no results for filter), error, long names, many items (20+ lines), demo vs live banner. Interactions: filter → table update; row → detail; drawer open/close; approve/reject confirm; stepper advance (valid next step only). Prototype the flow Overview → Orders → Order detail → advance status.
12	Acceptance criteria	Every number on A2 matches `/analytics/summary` for the same range. Order actions in Live produce the same results as today's pages. Invalid transitions are not offered. Customer pages show "planned endpoint" in Live until B1 ships. Tables are usable at 390 px (card mode).
13	Testing	Live against a **local test database** with fake WhatsApp credentials, or watch API logs; never test approve/notify against real customers. Integration tests for B1 in `SellBotLk.Tests` (same `TestApp` pattern). Demo: rule tests for `demo/rules.ts`.
14	Risks	Actions in Live send real WhatsApp messages (mitigation: confirm dialogs + test DB). Customer stats wrong if B2 is skipped (mitigation: compute from orders in B1). No server pagination (fine at the current scale; B8 later).
15	Estimate	7–10 dev days (incl. about 2 for B1, B2, B12) · Figma 4–6 days
16	Completion checklist	☐ A2 ☐ A3 ☐ A4 ☐ A5 ☐ A6 ☐ A7 ☐ (A8) ☐ B1 ☐ B2 ☐ B12 ☐ fixtures ☐ responsive card tables ☐ confirm dialogs
17	Before moving on	Order and payment badges and the stepper are final (Phase 4 reuses them). Demo customers and orders are stable, because conversations in Phase 3 reference them by ID.

PHASE 3 — WhatsApp Inbox and AI Experience

#	Item	Detail
1	Objective	Make the WhatsApp-first nature of the product obvious and honest: what customers say, what the AI understands, and what the code does
2	Expected outcome	A9 Inbox (demo, fully annotated), R2 Journey player, A10 AI Playground (Live cards for search, visual search and zones; parse after B5)
3	Pages	A9, R2, A10

#	Item	Detail
4	Main UI sections	Conversation list with language/state filters; chat thread with bubbles, media, interactive-button rendering, system events; context panel (customer, state, negotiation card, open order, AI trace); scenario picker + pipeline steps; playground cards with JSON viewer
5	Components	ConversationList(+Item), ChatThread, MessageBubble (text, image, document, interactive, typing), SystemEventRow, IntentChip (10 intents), LanguageChip, ConfidenceMeter, AITracePanel, NegotiationCard, DemoComposer, SimulationBanner, PhoneFrame, PipelineStep, PlaybackControls, PlaygroundCard, JsonViewer, ImageDropzone, ScoreBar, WhatsAppPreviewBubble
6	Backend APIs reused	`POST /products/search` (returns `whatsAppMessage` preview), `POST /products/visual-search`, `POST /delivery-zones/check`
7	Frontend tasks	Conversation fixtures: scenarios A–C from 5.6 + photo search + slip mismatch + duplicate slip + reorder + complaint + Gemini fallback. Each message has `{lang, intent, confidence, source: template	ai, sideEffects[]}`. Journey player plays a scenario step by step, each step linked to its R3 diagram node. Playground with Live/Demo implementations. Show "English template in current build" notes and disabled Track/Cancel buttons until B7. RTL is **not** needed (Sinhala and Tamil are left-to-right) but **fonts must cover Sinhala and Tamil scripts**: test glyph rendering.
8	Backend changes	**B5** `POST /api/v1/ai/parse` (admin-only, no side effects) [+ optional negotiation preview]. **B3** `GET /api/v1/conversations` (state, last activity, masked customer). **B7** handle `interactive` button replies (track and cancel with ownership + unpaid checks). Optional **B4** message log (privacy review first).
9	Dependencies	Phase 1 components; Phase 2 fixtures (customers, orders, products)
10	Pinterest keywords	"WhatsApp business inbox dashboard", "customer support shared inbox three panel", "chat app dashboard conversation list", "chatbot conversation annotation UI", "AI playground interface", "LLM testing console UI", "phone mockup chat explainer website", "process flow step animation web"
11	Figma requirements	Inbox: desktop 3-pane, tablet 2-pane (context panel as drawer), mobile list → thread. Message bubble variants (in/out × text/image/document/interactive × with/without annotations), system event row, typing indicator. States: no conversation selected, empty filter, AI fallback message, long Sinhala/Tamil text wrapping, image loading. Journey: scenario tabs, step states (pending/active/done/partial). Playground: idle, loading (Gemini latency is seconds, so design a meaningful loader), result, error (AI unavailable), Live vs Demo notice.
12	Acceptance criteria	Every bot message in fixtures exists as a template in code (spot-check list kept in the repo). Every simulated surface carries the simulation banner. Playground Live calls send **no** WhatsApp messages (verify in API logs). Sinhala and Tamil render correctly in Chrome, Safari, Firefox and on mobile.
13	Testing	Fixture validation test (each message has the required annotations). Visual check of si/ta glyphs. B5/B3/B7 integration tests in `SellBotLk.Tests` (B7: someone else's order number cannot be cancelled, mirroring `OrderPrivacyTests`).
14	Risks	Presenting simulation as live (mitigation: banners, wording "simulated"). B4 stores personal data (mitigation: optional, retention policy, masking, owner consent). Gemini cost or latency in Live playground (mitigation: admin-only, rate limit, existing 10-minute cache).

#	Item	Detail
15	Estimate	8–12 dev days (incl. about 2–3 for B3, B5, B7); +3–5 if B4 · Figma 4–6 days
16	Completion checklist	☐ A9 demo ☐ 8+ scenarios ☐ R2 ☐ A10 live cards ☐ B5 ☐ B3 ☐ B7 ☐ si/ta fonts verified ☐ banners
17	Before moving on	The chat components are stable (R2 and R1 reuse them). Honesty review: walk every demo screen and confirm nothing implies a live connection.

PHASE 4 — Business Workflows and Analytics

#	Item	Detail
1	Objective	Show the operational workflows that make the system a real business tool: payment verification, fulfilment, reporting, OCR, integrations
2	Expected outcome	A11 Payment review queue, A12 Delivery board + Zones, A13 Analytics, A14 Documents & OCR, A15 Integrations
3	Pages	A11, A12, A13, (A14), (A15)
4	Main UI sections	Review queue + slip viewer + extracted-vs-expected comparison; kanban columns with valid-transition actions; zone table + fee checker; date-ranged charts; upload card + extracted table; integration cards with status
5	Components	QueueList, SlipViewer (zoom/rotate), ComparisonTable, RejectReasonDialog, KanbanColumn, DeliveryCard, AdvanceStatusDialog, ZoneTable, FeeChecker, DateRangeBar, ChartCard, AreaChart, BarChart, DonutChart, Dropzone, ExtractedTable, IntegrationCard, StatusDot
6	Backend APIs reused	Approve/reject/slip endpoints, `PUT …/delivery-status`, `GET /delivery-zones`, `POST /delivery-zones/check`, `GET /analytics/summary`, `GET /documents`, `GET /documents/{id}`, `POST /documents/process`, `GET /health`
7	Frontend tasks	Payment queue from `GET /orders` filtered client-side. Keyboard shortcuts for approve (A) and reject (R) with confirmation. Board with only valid next steps (drag or button). Move the existing Zones page into A12. Migrate the Dashboard charts into A13. Documents upload with a "this changes stock" warning. Integration cards from B6. Demo: generated slip SVGs, sample invoice extraction, sample PDF invoice preview (a static demo PDF made with fictional data).
8	Backend changes	**B6** system status + DB health check (the referenced `AspNetCore.HealthChecks.MySql` package can be used; no new dependency). **B10** documents DTO + masking. Optional: expose slip extraction on the order DTO; `GET /orders/{id}/invoice` to download the PDF.
9	Dependencies	Phase 2 (badges, tables, stepper). Not Phase 3.
10	Pinterest keywords	"payment verification dashboard", "receipt review approve reject UI", "document review side by side UI", "delivery management kanban board", "logistics order tracking dashboard", "sales analytics dashboard date range", "OCR invoice extraction UI", "integrations page SaaS cards", "system status page design"
11	Figma requirements	A11: queue empty ("All caught up"), slip loading, PDF slip vs image slip, mismatch highlight, approve success toast, reject dialog. A12: columns at 1440/834 (horizontal scroll) and mobile (status tabs instead of columns), card with/without area, advance dialog. A13: charts with zero data, single day, 90 days. A14: upload progress, extraction result, partial match ("2 products not found"). A15: configured / not configured / unreachable.
12	Acceptance criteria	Approve/reject in Live behave exactly as the existing tested endpoints. The board never offers an invalid transition. Analytics numbers match the API. No raw slip sender names are visible without masking. The integrations page never shows a secret.
13	Testing	Payment and delivery flows on a test DB with faked WhatsApp. Chart rendering with edge datasets. B6 integration test (returns booleans only and no secret values; assert on response shape). B10 test that the DTO omits sensitive fields.
14	Risks	Approving in Live sends real invoices (mitigation: confirm dialog + test DB). Large slip images slow the queue (mitigation: lazy load on select). Charts misleading with few data points (mitigation: empty and low-data states).
15	Estimate	6–9 dev days (incl. about 1–2 for B6, B10) · Figma 3–5 days

#	Item	Detail
16	Completion checklist	☐ A11 ☐ A12 board ☐ A12 zones ☐ A13 ☐ (A14) ☐ (A15) ☐ B6 ☐ B10 ☐ demo slips/invoice
17	Before moving on	All admin pages are feature-complete. The demo dataset tells one coherent story (the same orders appear in Overview, Payments, Delivery and Analytics).

PHASE 5 — Recruiter Showcase and Production Polish

#	Item	Detail
1	Objective	Package the work so a recruiter understands it in 2–5 minutes, and make the app accessible, fast and safely deployable
2	Expected outcome	R1 Landing, R3 Architecture, R4 API reference, R5 Security & Reliability; guided-tour entry points; accessibility and performance pass; public demo deployment; documented Live deployment steps
3	Pages	R1, R3, (R4), R5; polish of all pages
4	Main UI sections	Hero + entry tiles; interactive architecture diagram; state machines; stack table; endpoint catalogue; guarantee → mechanism → test table; limitations; "Take the 3-minute tour" path
5	Components	Hero, EntryTile, TechBadge, StepStrip, DiagramCanvas (SVG hotspots), SectionNav, StateMachineDiagram, CodeBlock, EndpointRow, MethodBadge, GuaranteeCard, TestSuiteTable, TourStepper (optional "next" links between pages), Footer
6	Backend APIs reused	None (static content), except Live builds
7	Frontend tasks	Write the content from Sections 2–3 (keep claims inside the verified inventory). Draw diagrams as inline SVG. Tour links (R1 → R2 → A9 → A11 → A2 → R3 → R5). Accessibility pass (WCAG 2.2 AA: contrast, focus order, landmarks, labels, chart text alternatives, reduced motion for the Journey player). Performance: route-level code splitting (`React.lazy`) so Recharts and the chat bundle load only where used; image sizing; Lighthouse ≥ 90 on R1. SEO/meta for R1 (title, description, Open Graph image). Pre-deploy secret/URL scan of `dist/`.
8	Backend changes	**B11** (for any non-local Live deployment): CORS origins from configuration, `UseForwardedHeaders` (fixes the per-IP limiter behind a proxy), HTTPS, decide on product images (`UseStaticFiles` or external URLs), optional Dockerfile. Move the Gemini key from the query string to a header (small hardening). The README also needs updating (counts, button replies, multilingual scope).
9	Dependencies	Phases 1–4 (content references real pages)
10	Pinterest keywords	"SaaS landing page hero with dashboard mockup", "developer case study portfolio page", "system architecture diagram website", "technical documentation website design", "API reference page design", "security trust page SaaS", "interactive product tour UI"
11	Figma requirements	PresentationLayout pages at 3 breakpoints; diagram frames with hotspot states; Open Graph image (1200×630); focus states and reduced-motion variants of animated components; final redlines and handoff annotations.
12	Acceptance criteria	A first-time viewer can follow R1 → tour in ≤ 5 minutes. All claims trace to Section 3. Lighthouse (R1, A2): Performance ≥ 90, Accessibility ≥ 95, Best Practices ≥ 95. Demo `dist/` contains no API host, token or real data. Works at 390, 834 and 1440 px.
13	Testing	Lighthouse + axe browser extension; keyboard-only run of the tour; cross-browser (Chrome, Edge, Safari, Firefox); mobile device check; `grep` scan of `dist/`; optional Playwright smoke test of the tour in demo mode.
14	Risks	Overclaiming in copy (mitigation: link each claim to a file or test). Diagram drift from code (mitigation: date-stamp + "as of commit"). Demo deploy accidentally built in Live mode (mitigation: CI job that builds demo and fails if `VITE_API_BASE_URL` is set).
15	Estimate	6–9 dev days (incl. about 1–2 for B11) · Figma 3–4 days
16	Completion checklist	☐ R1 ☐ R3 ☐ (R4) ☐ R5 ☐ tour links ☐ a11y pass ☐ Lighthouse ☐ code splitting ☐ OG/meta ☐ dist scan ☐ demo deployed ☐ README updated ☐ B11 (if Live hosted)
17	Done criteria	A recruiter link and a 2–5-minute walkthrough (Section 13.12) rehearsed end to end

9.2 Effort summary

Phase	Dev days	Figma days
1 Foundation	5–7	3–5
2 Business admin	7–10	4–6
3 WhatsApp & AI	8–12 (+3–5 with B4)	4–6
4 Workflows & analytics	6–9	3–5
5 Showcase & polish	6–9	3–4
Total	**32–47** (MVP about 20–26)	**17–26**

10. Pinterest → Figma → Frontend Workflow

10.1 Process

1. **Collect** 5–8 references per page using the keywords below. Save them on one Pinterest board per *layout* (Admin, Presentation), with sections per page.
2. **Pick one visual direction** for the whole product: one reference for the admin shell and one for the presentation pages. Other references contribute *patterns*, not styles.
3. **Annotate** each kept pin with what you are borrowing (for example "table density", "chat bubble meta row").
4. **Figma:** Foundations → components → templates → pages (Phase order). Rebuild, never trace. Use your own tokens.
5. **Handoff:** every frame lists its route, data source (`data.*` method), states covered and responsive rules.
6. **Implement** one phase at a time. Compare against the Figma frames at all 3 breakpoints before closing the phase.

10.2 What to look for in references

Element	Look for	Avoid
Layout	240–280 px sidebar with grouped sections; content max-width for reading pages; clear page header with actions on the right	Dashboards that only work at 1920 px
Sidebar	Group labels, count badges, collapsed icon rail, active state that doesn't rely on colour alone	Deep nested menus (SellBotLK has 13 admin destinations)
Header	Page title + breadcrumb on detail pages, global search slot, mode/user menu	Large decorative hero headers inside the admin
Cards	KPI cards with label, value, delta/sub-text; "needs action" cards with a list + CTA	Cards with charts too small to read
Tables	Dense but readable rows, sticky header, sortable columns, status badges, row hover + row click, responsive card fallback	Tables that hide status behind icons only
Charts	Area chart with tooltip, horizontal bar for top products, donut with legend; consistent number formatting (LKR)	3D charts, gradients that hurt readability
Filters/search	Tab + chip combinations, filters visible above the table	Filters hidden in modals
Forms	Grouped fields, inline validation, numeric inputs with LKR prefix, side drawer editing	Long single-column forms for 13 product fields without grouping
Chat	WhatsApp-like bubbles **without copying WhatsApp's brand**; meta row under bubble (time, chips); system events centred; media thumbnails	Pixel copies of WhatsApp UI or logos
Conversation layout	3-pane inbox with a collapsible context panel	Inboxes without a selected-state design
Detail panels	Drawer + full-page variants; key-value lists; timelines	Modal-only details (bad on mobile)
Status indicators	Badge sets with 6+ distinct states that work in greyscale (icon + text)	Colour-only dots
Empty states	Short explanation + action ("No payments waiting — you're all caught up")	Large illustrations everywhere
Responsive	References that show mobile versions of tables and inboxes	Desktop-only shots

10.3 Shared vs separate layouts

- **Admin layout (shared):** A2–A15 (A1 Login is a standalone centred page).
- **Presentation layout (separate):** R1, R2, R3, R4, R5. R2 embeds chat components from the admin design system so both layouts feel like one product.

10.4 Per-page Pinterest and Figma guide

Page	Pinterest search phrases	Key elements to find	Figma frames (D=1440, T=834, M=390)	States to design
A1 Login	"minimal SaaS login page", "admin sign in split screen", "B2B login card"	Centred card, error message placement, password toggle	D, T, M	default, error, rate-limited (429), loading
A2 Overview	"e-commerce admin dashboard overview", "operations dashboard needs attention", "KPI cards dashboard minimal"	KPI row, action queues, compact recent table	D, T, M	loading, empty (new business), normal, demo banner
A3 Products	"inventory management table thumbnails", "product catalog admin grid list toggle", "stock level indicator UI"	Thumb + name cell, stock meter, filter bar, grid/list toggle	D, T, M (card list)	loading, empty, no results, low-stock rows, inactive
A4 Product detail	"product edit side drawer", "pricing tiers card UI", "inventory history timeline"	Drawer with tabs, price ladder, form groups	D drawer, M full page	view, edit, validation error, save success, delete confirm
A5 Orders	"order management dashboard filters", "orders table status tabs", "e-commerce orders admin badges"	Status tabs + payment chips, badges, countdown chip	D, T, M (cards)	loading, empty, filtered empty, long list (pagination)
A6 Order detail	"order details page admin", "order timeline stepper", "payment verification panel"	Two-column detail, stepper, payment card with slip	D, T, M	unpaid, pending review, paid, cancelled, delivered, action error
A7 Customers	"CRM customer list table", "customer directory dashboard"	Language chip, masked phone, value columns	D, T, M	loading, empty, Live "endpoint planned"
A8 Customer detail	"CRM customer profile page", "customer 360 view"	Profile header + KPIs + orders table	D, M	no orders, many orders
A9 Inbox	"WhatsApp business inbox dashboard", "customer support shared inbox three panel", "chat inbox conversation list UI", "chatbot conversation annotations"	3 panes, bubble meta, system events, context panel	D (3-pane), T (2-pane), M (list + thread)	none selected, typing, image, slip, AI fallback, long si/ta text, simulation banner
A10 AI Playground	"AI playground interface", "LLM prompt testing console", "image analysis result UI"	Input/output split, JSON viewer, score bars	D, T, M	idle, loading, result, AI error, Live vs Demo notice
A11 Payments	"payment verification dashboard", "receipt review approve reject", "document review side by side"	Queue + viewer + comparison table	D, T, M	empty queue, image slip, PDF slip, amount match, reject dialog
A12 Delivery	"delivery management kanban", "logistics order tracking dashboard", "delivery zones pricing table"	Columns with counts, card density, tabs	D, T (scroll), M (tabs)	empty column, no area, advance dialog, zones checker result
A13 Analytics	"sales analytics dashboard date range", "revenue chart dashboard card"	Date bar, chart cards, legends	D, T, M	zero data, short/long range
A14 Documents	"OCR invoice extraction UI", "file upload dropzone results table"	Dropzone, extracted table, notes	D, M	uploading, extracted, partial match, failed
A15 Integrations	"integrations page SaaS cards", "system status page"	Status cards, key facts	D, M	configured, not configured, unreachable

Page	Pinterest search phrases	Key elements to find	Figma frames (D=1440, T=834, M=390)	States to design
R1 Landing	"SaaS landing page hero dashboard mockup", "developer case study page", "AI product landing page"	Hero, 3-step strip, entry tiles, badges	D, T, M	default; OG image frame
R2 Journey	"chatbot flow visualization", "phone mockup chat with explanation", "process step animation web"	Split phone + steps, playback controls	D, T, M (stacked)	step pending/active/done/partial, reduced motion
R3 Architecture	"system architecture diagram website", "technical case study design", "software architecture infographic"	Diagram with hotspots, sticky section nav	D, T, M	hotspot selected
R4 API reference	"API documentation website design", "developer docs endpoint list"	Grouped endpoints, method badges, code blocks	D, M	expanded/collapsed endpoint
R5 Security	"security trust page SaaS", "test coverage report UI", "layered defense diagram"	Guarantee cards, test table, limitations	D, T, M	—

10.5 Developer handoff requirements (all pages)

- Every component in Figma has a matching code name (for example `OrderStatusBadge`) and lists its props and variants.
- Every page frame lists its route, `data.*` calls, states covered, breakpoint behaviour and keyboard interactions.
- Enumerations designed exactly as in code: OrderStatus (Pending*, Confirmed, Processing, Dispatched, Delivered, Cancelled; FraudPending hidden), PaymentStatus (Unpaid, PendingReview, Paid, Refunded*), ConversationState (Greeting, Browsing, Ordering, Negotiating, Support, Completed*), Intent (10 values), Language (en, si, ta). Values marked * exist in the enum but are never set by current code. Design a badge, but expect it not to appear.
- Number format: `LKR 28,500` (code uses `N0`). Dates: local, `dd MMM yyyy` (as on the invoice).
- Fonts must include Sinhala and Tamil glyph coverage (check before committing to a typeface).
- Motion specs for the Journey player and typing indicator, plus reduced-motion alternatives.

11. Backend Changes That May Be Necessary

None of these were implemented in this investigation. Sizes: **S** ≤ 1 day, **M** 1–3 days, **L** 3+ days.

ID	Change	Why	Phase	Size	Required?
B1	`GET /api/v1/customers?search&language&page` and `GET /api/v1/customers/{id}`: DTO with **computed** order count, spend and last order from `Orders` (excluding cancelled), plus a masked phone option	No customers API exists (G2)	2	S–M	For Live A7/A8
B2	Maintain `Customer.TotalOrders`, `TotalSpend` and `LastOrderDate` on create, cancel and approve (or drop them and always compute)	Counters are never written; the returning-customer greeting is dead (G3)	2	S	Recommended
B3	`GET /api/v1/conversations`: customer (masked), state, last activity, has active negotiation	Live inbox list	3	S	For Live A9 list
B4	`ConversationMessages` table (direction, type, text/caption, wamid, intent, language, confidence, createdAt) written by the webhook and `WhatsAppSendService`; `GET /api/v1/conversations/{id}/messages`; retention purge in `MaintenanceService`	Live transcripts (G1). **Stores personal data**: needs a privacy decision.	3	M–L	Optional
B5	`POST /api/v1/ai/parse` (admin, rate-limited): runs `GeminiService.ParseMessageAsync` with the catalogue and returns the parsed intent. No customer is created and no message is sent. Optional `POST /api/v1/ai/negotiation-preview` → `NegotiationService.EvaluateOfferAsync` (already side-effect free)	Live AI Playground	3	S	For Live A10 parse
B6	`GET /api/v1/system/status` (admin): booleans for WhatsApp, Gemini, owner phone and payment details configured; DB reachable; pending migrations; `UnpaidExpiryHours`; maintenance last run; build version. Add a DB health check to `/health` using the already-referenced MySQL health-check package	Integrations page; real health (G8)	4	S	For Live A15
B7	Handle inbound `interactive.button_reply` (`track_{no}`, `cancel_{no}`): extend `WhatsAppMessageItemDto`; track → existing status handler; cancel → only own + unpaid orders via `CancelOrderAsync(id, onlyIfUnpaid: true)`	Buttons are shown but do nothing (G4)	3	S–M	Recommended (journey honesty)
B8	Orders list: `paymentStatus` filter, search (number/customer), server pagination and sort	Scale; cheaper pending-review badge	2/4	S	Optional
B9	`GET /api/v1/products/{id}/inventory-logs`	Inventory history tab	2	S	Optional
B10	`GET /documents` returns `DocumentResponseDto` (not the entity); mask slip sender/receiver names	Data minimisation (Section 7.3)	4	S	For Live A14
B11	Deployment hardening: CORS origins from configuration, `UseForwardedHeaders`, HTTPS, static images decision, optional Dockerfile; Gemini key in a header instead of the query string	Any hosted Live deployment (G8)	5	S–M	If Live is hosted
B12	Retire or validate `PUT /orders/{id}/status` (apply `IsValidTransition` + notifications), or make the UI use `delivery-status` only	Two inconsistent status paths	2	S	Recommended
B13	Capture delivery area/address in chat, quote the fee via `CheckZoneAsync`, add the fee to totals	Product gap (G5)	—	M	Not needed for the showcase
B14	Localise the remaining English-only templates (confirmation, payment, delivery, image)	Multilingual gap (G6)	—	M	Not needed for the showcase; improves the demo story

12. Security and Privacy Considerations

1. **Two builds, two audiences.** Only the demo build is public. The Live build is never deployed to a public static host, and its API must not be exposed without B11.
2. **No real data in the repo or the demo.** Fixtures are fictional and hand-written. Do not commit screenshots of the production dashboard or WhatsApp.
3. **PII handling in Live:** phone numbers masked by default; slips loaded on demand; document extraction masked (B10); B4 transcripts only with a retention period and a reason to keep them.
4. **Secrets never reach the browser.** The status endpoint returns booleans. No settings page. JWT in `localStorage` is acceptable for a single-owner local tool, but for a hosted Live build prefer short expiry plus a re-login prompt, and add a strict Content-Security-Policy.
5. **Real side effects are explicit.** Every Live action that messages a customer (approve, reject, delivery status, cancel via expiry) gets a confirm dialog that says so. Do Live UI testing against a test database with faked WhatsApp credentials. The test suite already proves this pattern (`TestApp`).
6. **Playground cost and abuse.** B5 must be admin-only and rate-limited. It must not create customers or conversations.
7. **Accuracy of claims is a security property too.** Do not show fraud detection, blocked customers or button-reply handling as features.
8. **Pre-publish scan** of the demo `dist/` for API hosts, tokens, `94` + 9-digit numbers outside the fake range, and email addresses.

13. Final Recommendations

13.1 Overall website structure

One React app (`sellbotlk-admin`, renamed in the UI to **SellBot.lk Console**) with an **Admin layout** (13 business pages) and a **Presentation layout** (5 showcase pages), built in two modes: public **Demo** and private **Live** (Sections 6–7).

13.2 Navigation and sidebar

See Section 6.3. Groups: Overview · Commerce (Orders, Payments, Delivery, Products, Customers) · WhatsApp & AI (Inbox, AI Playground) · Insights (Analytics, Documents) · System (Integrations) · About this project (demo only).

13.3 Complete frontend page list

ID	Page	Layout	Mode	Priority	Phase
A1	Login	Standalone	L	P0	1
A2	Overview	Admin	L+D	P0	2
A3	Products	Admin	L+D	P0	2
A4	Product detail (drawer)	Admin	L+D	P1	2
A5	Orders	Admin	L+D	P0	2
A6	Order detail	Admin	L+D	P0	2
A7	Customers	Admin	D (L after B1)	P1	2
A8	Customer detail	Admin	D (L after B1)	P2	2
A9	WhatsApp Inbox	Admin	D (L after B3/B4)	P0	3
A10	AI Playground	Admin	L+D	P1	3
A11	Payment review	Admin	L+D	P0	4
A12	Delivery (Board + Zones)	Admin	L+D	P0	4
A13	Analytics	Admin	L+D	P1	4
A14	Documents & OCR	Admin	L+D	P2	4
A15	Integrations & status	Admin	D (L after B6)	P2	4
R1	Recruiter landing	Presentation	D	P0	5
R2	Customer journey player	Presentation	D	P0	3
R3	Architecture & stack	Presentation	Static	P0	5
R4	API reference	Presentation	Static	P2	5
R5	Security, reliability & testing	Presentation	Static	P0	5

13.4 Pages to design first in Figma

1. Foundations + component sheet (badges for every status enum, table, card, buttons, inputs)
2. AdminLayout shell at 1440 / 834 / 390
3. **A5 Orders** (proves table, filter, badge and responsive card patterns)
4. **A6 Order detail** (proves detail layout, stepper and payment panel)

5. **A9 Inbox** (the signature screen; proves chat components early, since they are the riskiest design)
6. **R1 Landing** (sets the presentation layout tone)
7. Login

13.5 Exact five-phase roadmap

Phase 1 Foundation → Phase 2 Business admin → Phase 3 WhatsApp & AI → Phase 4 Workflows & analytics → Phase 5 Showcase & polish. Full details in Section 9.

13.6 Pinterest keywords for the first design stage

"SaaS admin dashboard sidebar navigation" · "dashboard design system components" · "admin dashboard UI kit tables badges" · "order management dashboard table filters" · "order details page admin timeline" · "customer support shared inbox three panel" · "WhatsApp business inbox dashboard" · "SaaS landing page hero dashboard mockup" · "minimal SaaS login page" · "status badge design system"

13.7 Backend changes

Required for Live parity: B1, B3, B5, B6, B10. Recommended: B2, B7, B12. Hosting: B11. Optional: B4, B8, B9. Product (not showcase): B13, B14. See Section 11.

13.8 Frontend architecture recommendation

Keep React 19 + React Router 7 + Vite 8 + TypeScript 6 + Recharts 3. Add no UI framework by default. Use plain CSS with design tokens (CSS custom properties), optionally CSS Modules per component (built into Vite, no dependency). Folder layout: `src/layouts`, `src/pages/{admin,showcase}`, `src/components/{ui,data-display,chat,charts,showcase}`, `src/data/{live,demo}`, `src/hooks`, `src/styles/tokens.css`. Use a `DataSource` adapter with build-time mode selection, route-level code splitting, and a shared `useAsync` hook. Consider a data-fetching library only if caching or invalidation becomes painful. It is not needed for this size.

13.9 Live Admin vs Recruiter Demo strategy

Build-time `VITE_DATA_MODE`. The demo bundle contains no API URL or live client. Fictional fixtures follow the real rules (a TypeScript mirror tested against the same values as the C# tests). Persistent banners and "Simulated" toasts. Live is private, requires login, and has confirm dialogs on every customer-facing side effect.

13.10 Security and privacy

Section 12.

13.11 Estimated total effort

32–47 developer days + 17–26 Figma days for the full scope. **MVP: about 20–26 developer days.**

13.12 Recruiter walkthrough (2–5 minutes)

Time	Screen	Say / show
0:00–0:30	R1 Landing	"Customers in Sri Lanka shop through WhatsApp. SellBotLK lets a furniture business sell, negotiate, take payments and deliver entirely in chat, and gives the owner a console to manage it. Fictional data, real architecture."

Time	Screen	Say / show
0:30–1:30	R2 Journey, scenario A (Sinhala)	Play "chair ekak denna": signature check → dedupe → Gemini detects Sinhala + Order intent → **code** checks stock and price → order created → confirmation. Point at a step card's file name.
1:30–2:15	A9 Inbox	Open the Tamil thread (language chips), then the negotiation thread: "The AI reads the offer; the price floor is enforced in code and covered by tests."
2:15–3:00	A11 Payments	Slip OCR result vs order total → Approve → "AI can never mark an order Paid; a slip can be used once; the invoice PDF is generated server-side."
3:00–3:30	A12 Delivery / A2 Overview	Advance an order and note that only valid transitions are offered and the customer is notified. KPIs and the queue.
3:30–4:30	R3 Architecture → R5 Security	Pipeline diagram; webhook defence layers; "63 integration tests on real MySQL prove concurrency, idempotency and money rules."
4:30–5:00	R5 limitations → GitHub	Honest limitations (transcripts not stored, buttons pending) → repository link.

13.13 Features that should NOT be built

Storefront/cart/checkout · customer accounts · Settings page with secrets · user/role management · Live agent reply composer · WhatsApp template manager · fraud center · daily reports · zone CRUD · Hangfire dashboard · voice-note demo · fake "real-time" counters · any Live chart (language or intent mix) not backed by stored data (Section 8.1).

14. Validation, Assumptions, Risks and Open Items

14.1 Validation performed for this report

Check	Result
Source read	All 6 controllers, the webhook controller, middleware, 13 services, 2 integrations, 11 entities, DbContext + seed, repositories, DTOs, the 5 frontend pages + API client + components, 3 test files, CI, manifests
Secrets	`appsettings.Development.json` exists locally and is git-ignored. It was read only with values redacted, and no values appear in this report. `.env` files were not read.
Tests	Read, **not executed** (they need MySQL). Pass status is NOT VERIFIED.
Live integrations	Meta and Gemini behaviour NOT VERIFIED (no live calls made, no messages sent)
Repository changes	Only `docs/showcase-investigation/` was created. Pre-existing uncommitted changes were left untouched (hash-checked before and after).
PDF	Generated from this Markdown with locally installed Microsoft Edge (headless print-to-PDF) via a temporary standard-library Python converter kept outside the repo. No packages were installed. Checked with PdfPig (already in the local NuGet cache): page count, a "Page X of Y" footer on every page after the cover, no glyphs outside the printable area, and Sinhala and Tamil glyphs present.

14.2 Incomplete or unverified features (consolidated)

- **NOT IMPLEMENTED:** message transcript storage; customers API; conversations API; interactive button-reply handling; blocked-customer enforcement; fraud detection; daily reports; Hangfire jobs; voice notes; CustomerBill documents; WhatsApp message templates; delivery area capture and fee in chat orders; area-specific delivery quote in chat; DB health check; static image serving; deployment configuration; frontend tests; inventory-log read endpoint; complaint escalation.
- **PARTIALLY IMPLEMENTED:** customer records (counters never updated); Sinhala/Tamil replies (several English-only templates); conversation management (state only); status callbacks (parsed, ignored); damage reports (admin upload only); generic order status endpoint (unvalidated); health endpoint (liveness only); input validation (mixed); AI free-text replies (fallback only).
- **NOT VERIFIED:** live Meta send/receive, typing indicator, media download and document upload; live Gemini parsing quality for si/ta/Singlish; the multi-turn negotiation "confirm" classification; test-suite pass status; CI status on the current branch.

14.3 Assumptions

1. One business owner operates the system (matches the single-admin authentication model).
2. The public showcase must never touch production. Recruiters only need the demo.
3. "Suhada Furniture" is the client name for prose. The product brand in the UI stays SellBot.lk unless you rename it in code.
4. Current data volumes are small (client-side filtering is acceptable). Revisit B8 if orders grow past a few thousand.
5. You will choose the visual direction. Nothing in this report fixes colours, fonts or themes, apart from the functional need for Sinhala and Tamil glyph support.
6. Estimates assume one developer working in focused days, excluding review and waiting time.

14.4 Key risks

Risk	Likelihood	Impact	Mitigation
Demo presented or perceived as a live WhatsApp integration	Medium	High (credibility)	Banners, "Simulated" wording, honesty review at the end of Phase 3
Live UI testing sends messages or invoices to real customers	Medium	High	Test DB + fake credentials; confirm dialogs
Demo fixtures contradict real rules	Medium	Medium	`demo/rules.ts` mirrored and tested (Section 7.4)
Storing transcripts (B4) creates a privacy liability	Medium	High	Keep B4 optional, minimise, set retention, mask
Scope creep from generic admin patterns	High	Medium	Section 8.1 "do not build" list; MVP table
Sinhala/Tamil rendering issues with the chosen font	Medium	Medium	Check glyph coverage in Phase 1
Docs drift from code	Medium	Low	Date and commit stamp on R3/R5; update the README in Phase 5
Public demo accidentally built in Live mode	Low	High	CI guard + `dist/` scan

Appendix A — Evidence Index

Topic	Primary files
Startup, DI, auth, rate limits, pipeline	`SellBotLk.Api/Program.cs`
Configuration aliases	`SellBotLk.Api/LegacyEnvConfiguration.cs`, `SellBotLk.Api/.env.example`
Webhook	`SellBotLk.Api/Webhooks/WhatsAppWebhookController.cs`, `Middleware/HmacVerificationMiddleware.cs`, `Models/DTOs/WhatsAppMessageDto.cs`
Conversation logic	`SellBotLk.Api/Services/MessageProcessingService.cs`, `Services/NegotiationService.cs`, `Models/NegotiationContext.cs`
AI	`SellBotLk.Api/Integrations/Gemini/GeminiService.cs`, `Integrations/GeminiVisionService.cs`, `Services/VisualSearchService.cs`, `Services/ProductService.cs`
Orders, stock, expiry	`Services/OrderService.cs`, `Data/Repositories/*.cs`, `Services/MaintenanceService.cs`
Payments, invoices	`Services/PaymentMatchingService.cs`, `Services/InvoiceService.cs`
Documents	`Services/DocumentService.cs`, `Controllers/DocumentsController.cs`
Delivery	`Services/DeliveryService.cs`, `Controllers/DeliveryZonesController.cs`
Analytics	`Services/AnalyticsService.cs`, `Controllers/AnalyticsController.cs`
Schema	`Data/AppDbContext.cs`, `Models/Entities/*.cs`, `Data/Migrations/*`
Frontend	`sellbotlk-admin/src/App.tsx`, `src/api/*.ts`, `src/components/*.tsx`, `src/pages/*.tsx`
Tests	`SellBotLk.Tests/SecurityAndReliabilityTests.cs`, `TestApp.cs`, `UnitTest1.cs`
CI	`.github/workflows/ci.yml`